SISTEMAS OPERACIONAIS

Memória virtual
e
Substituição de páginas

Prof. Matheus A. Souza
Eng. Software/Sist. Informação
PUC Minas

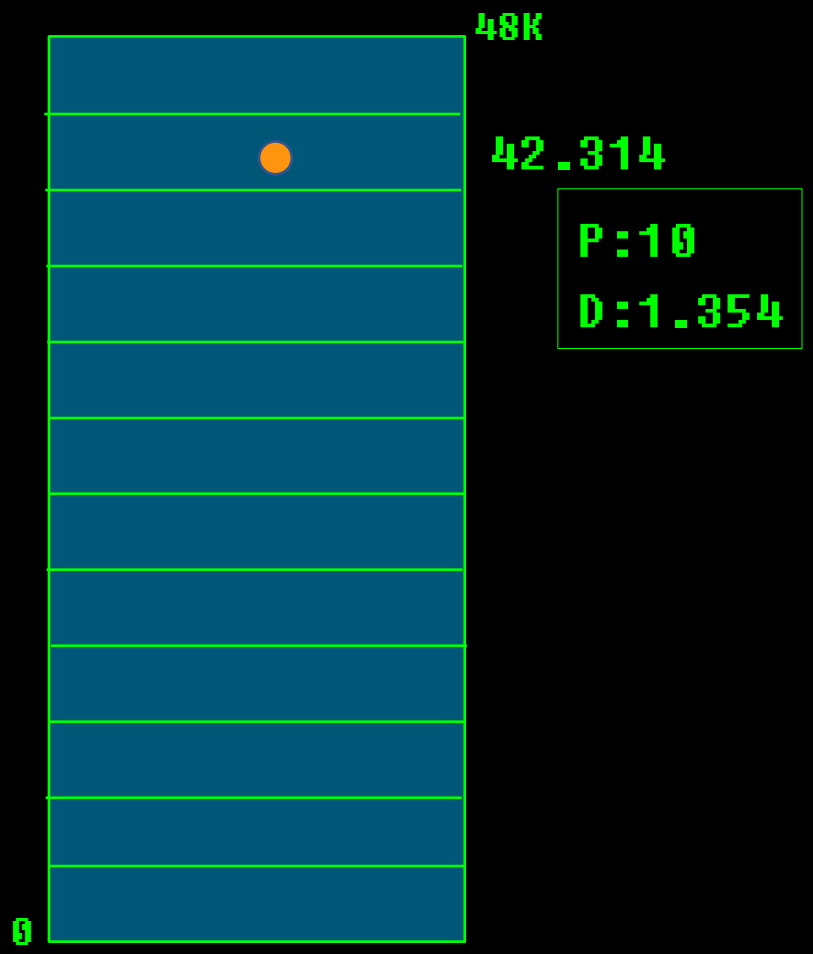
Memória virtual

- SO mantém em memória principal somente o necessário para a execução dos processos em um dado momento

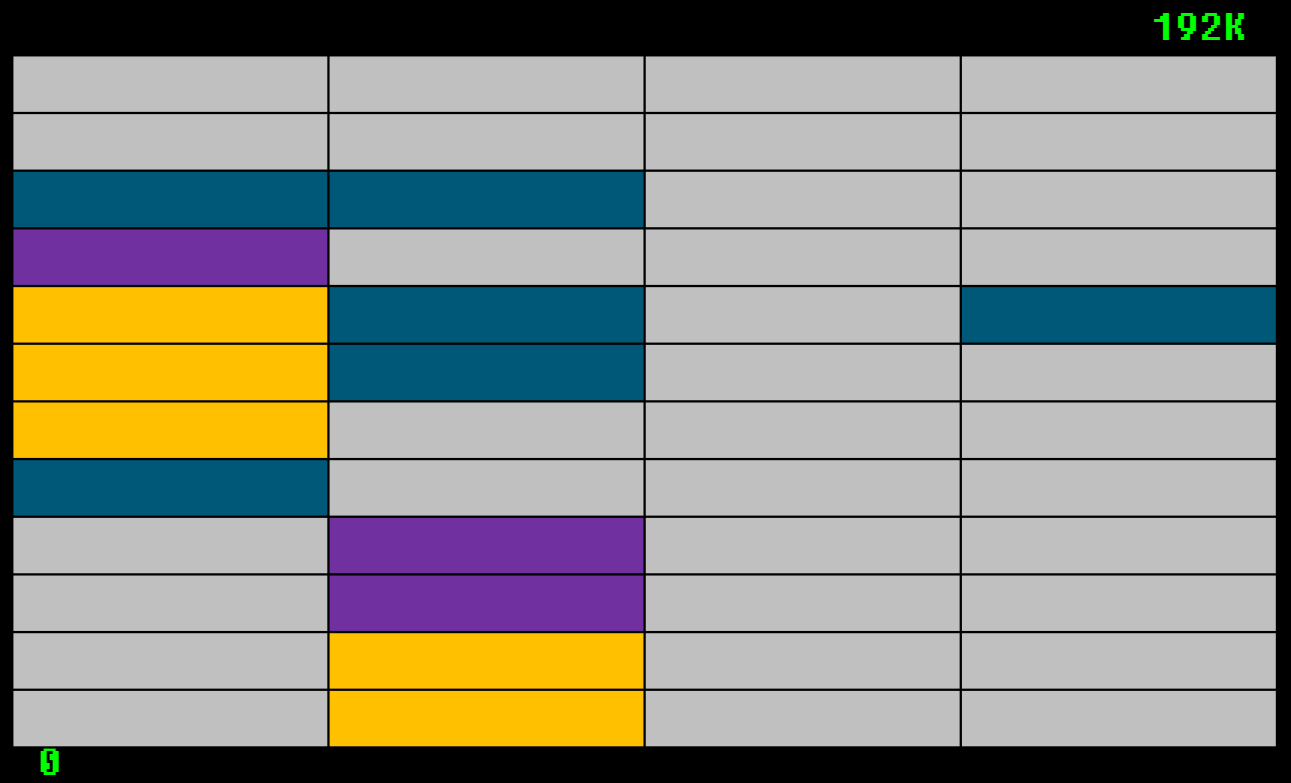
- Páginas são carregadas para memória principal de acordo com a demanda do processo

- Página não presente em memória principal: falta de página (*page fault*)

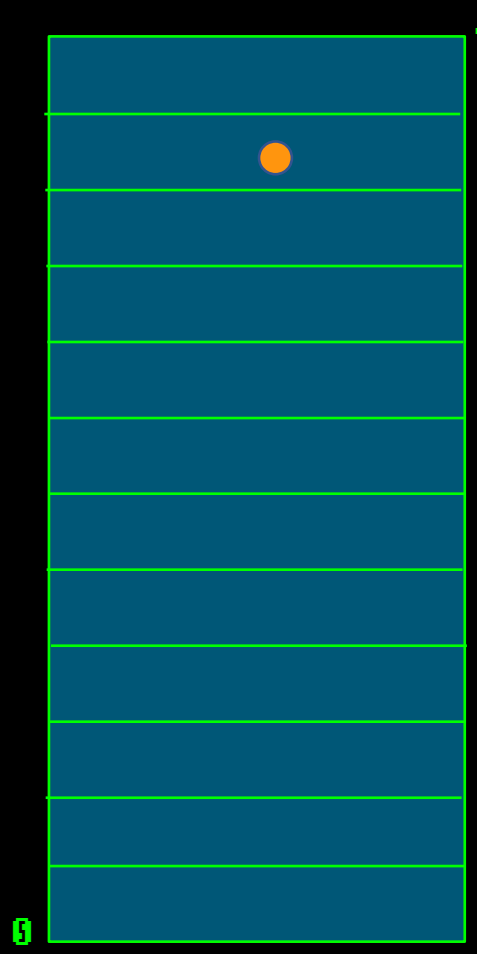
Paginação e memória virtual



11	–
10	–
9	–
8	43
7	–
6	18
5	9
4	–
3	–
2	19
1	4
0	21



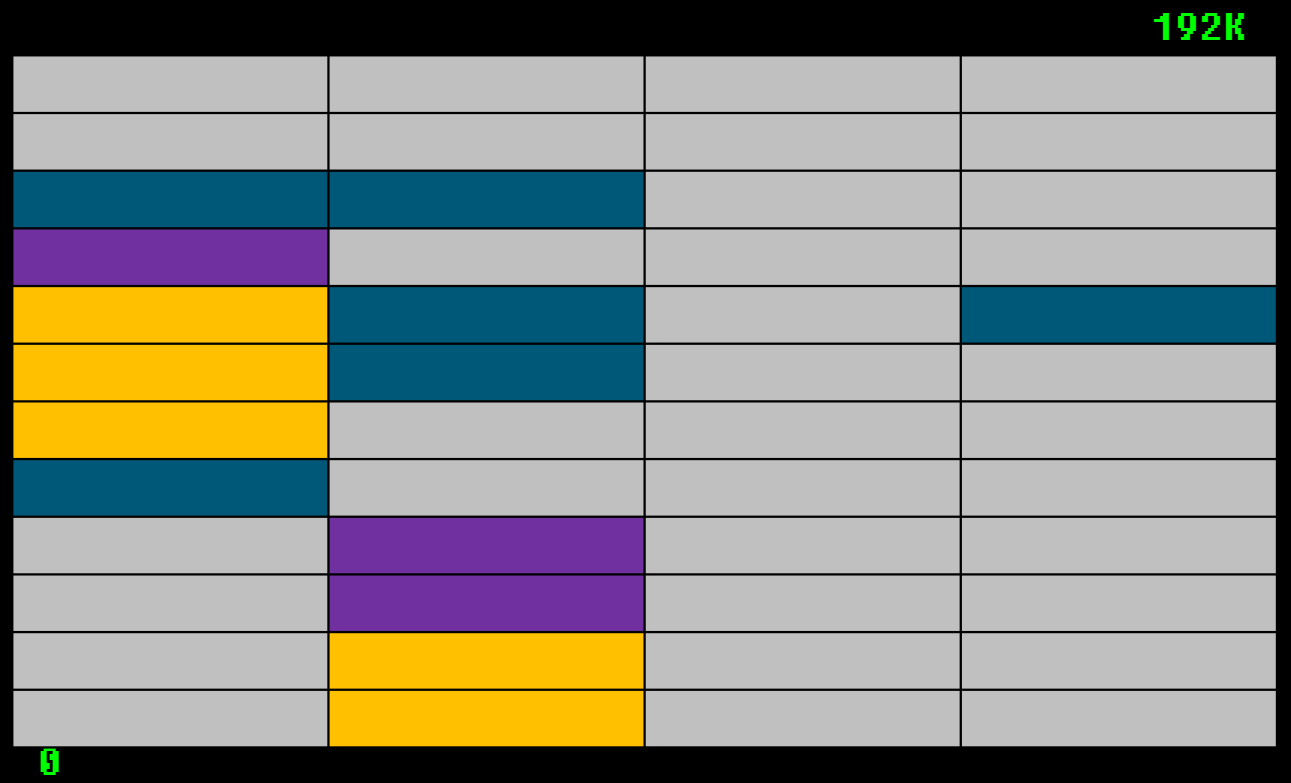
Paginação e memória virtual



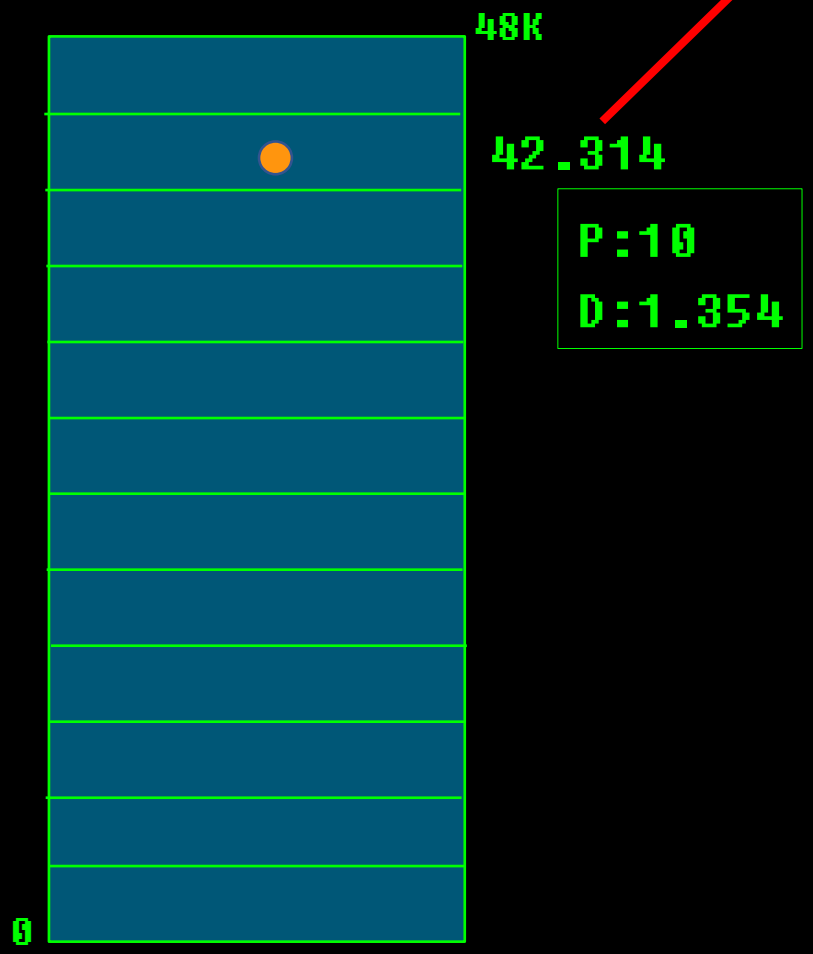
42.314

P:10
D:1.354

11	-
10	-
9	-
8	43
7	-
6	18
5	9
4	-
3	-
2	19
1	4
0	21



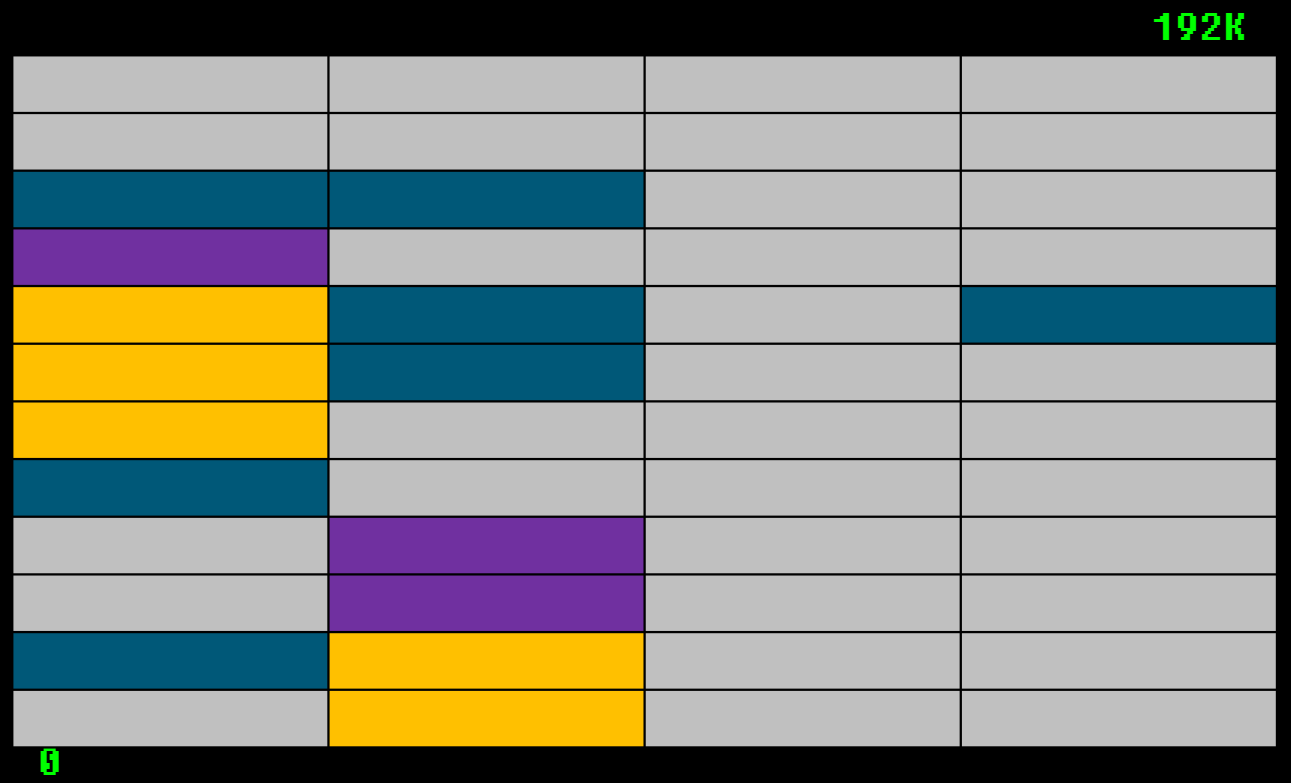
Paginação e memória virtual



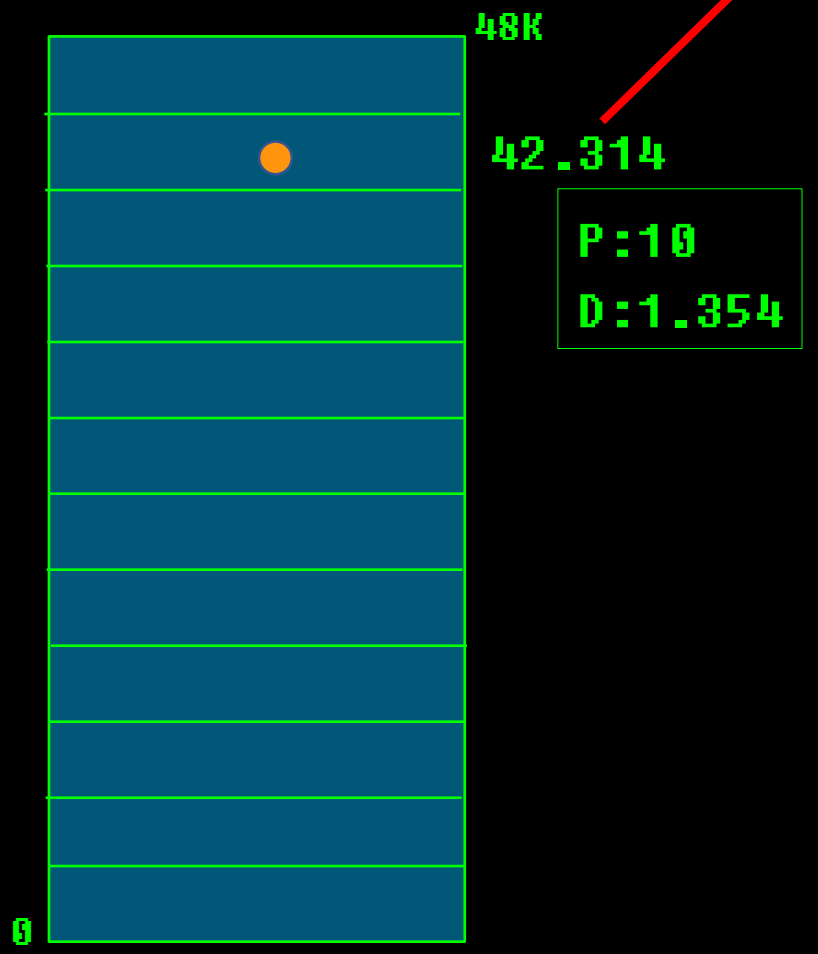
42.314

P:10
D:1.354

11	-
10	1
9	-
8	43
7	-
6	18
5	9
4	-
3	-
2	19
1	4
0	21



Paginação e memória virtual

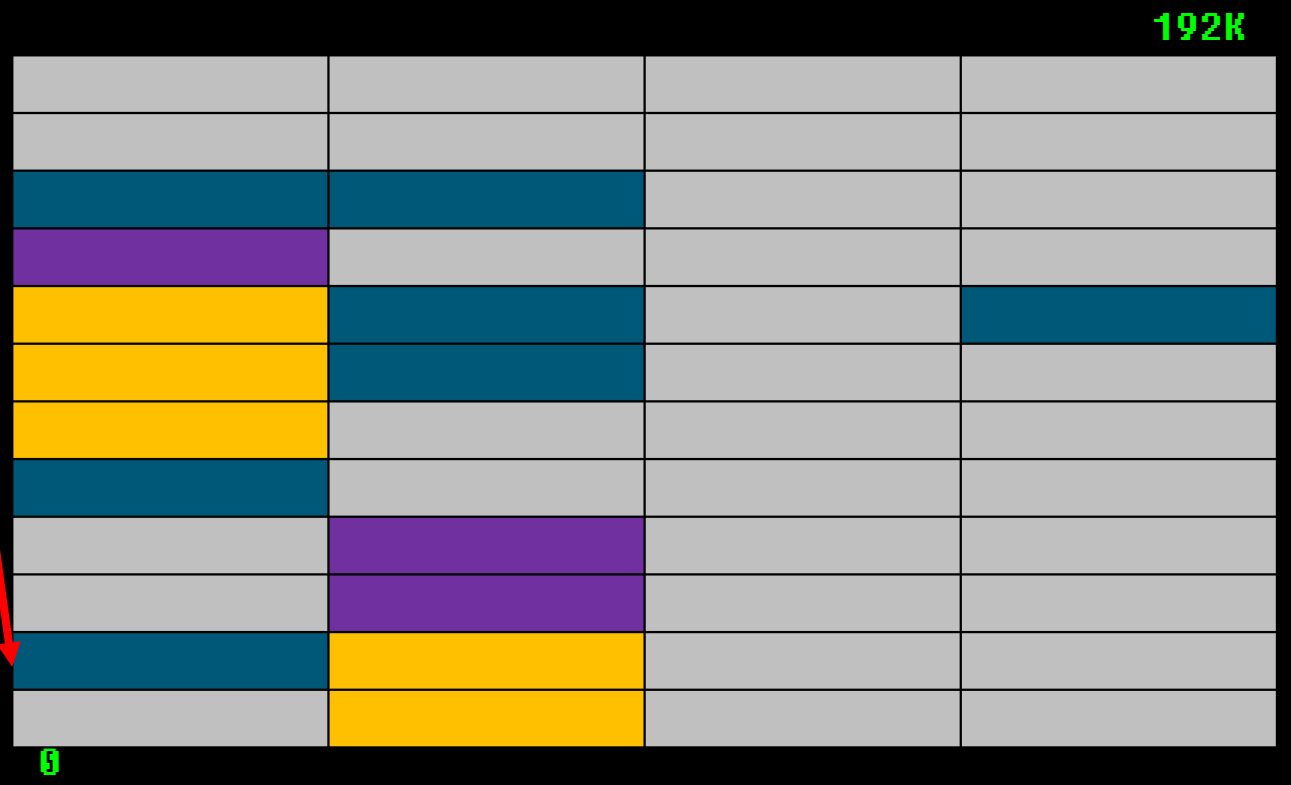


42.314

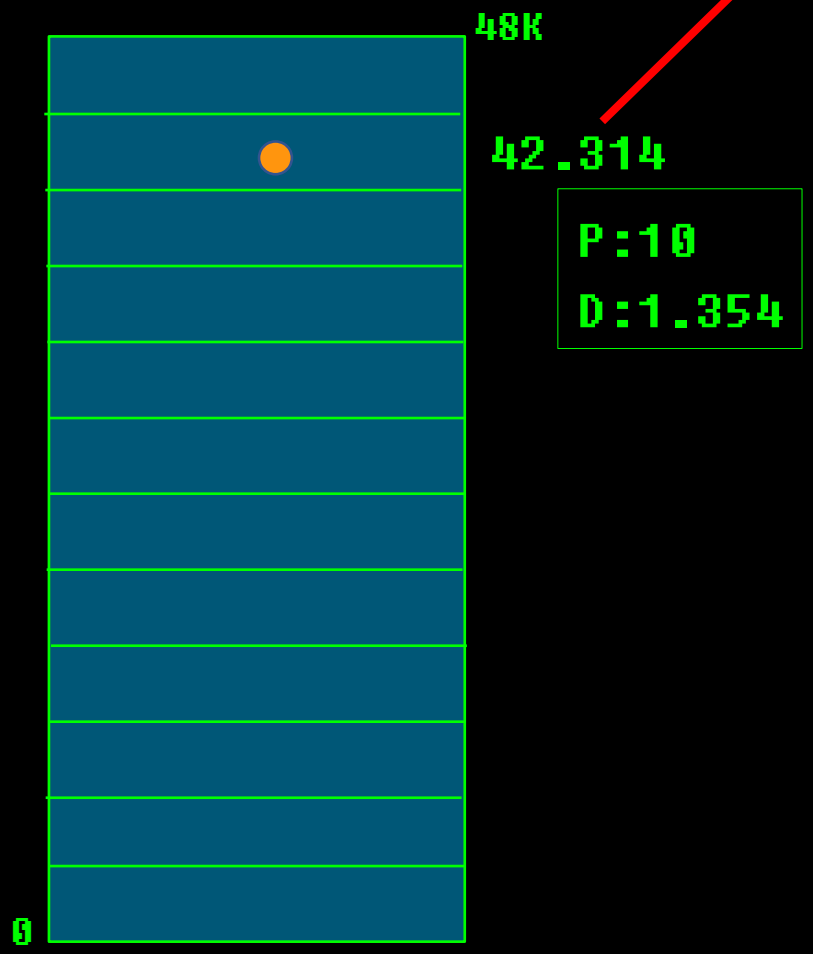
P:10

D:1.354

11	-
10	1
9	-
8	43
7	-
6	18
5	9
4	-
3	-
2	19
1	4
0	21



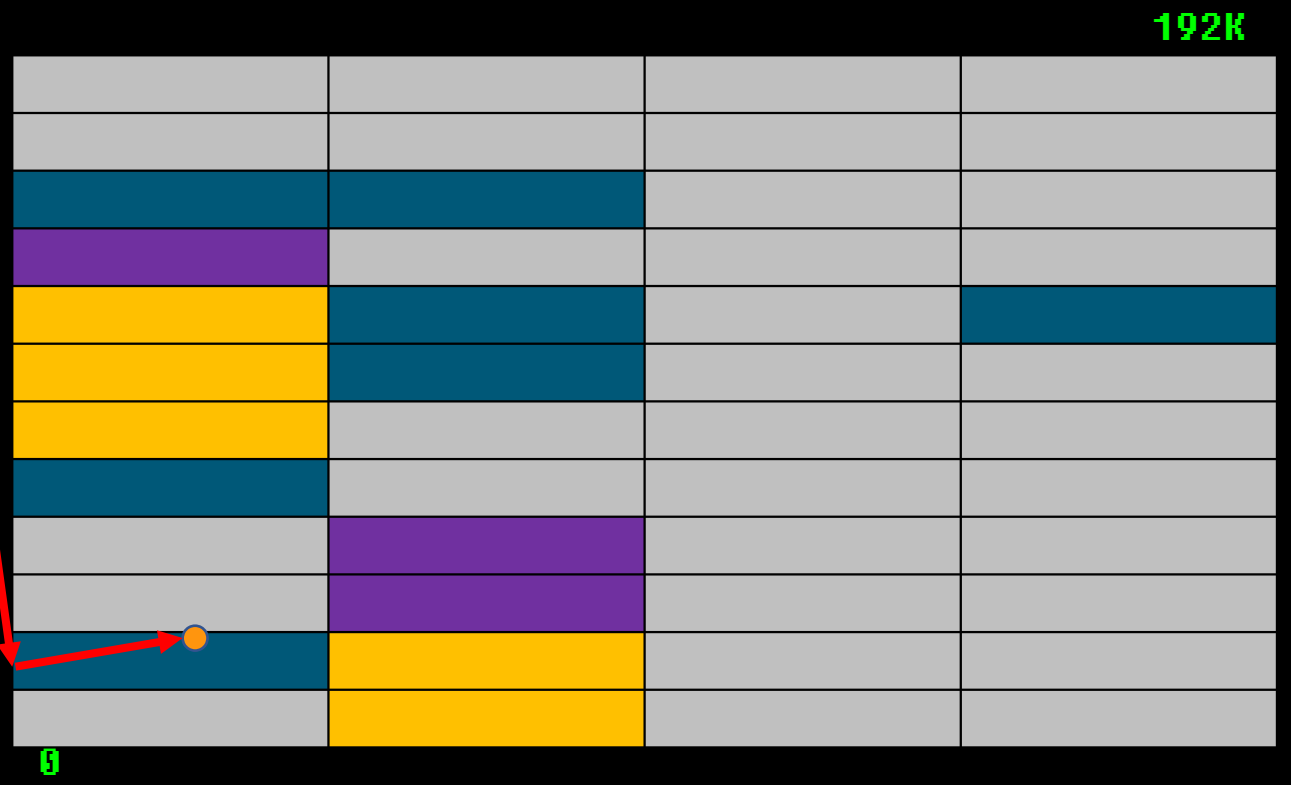
Paginação e memória virtual



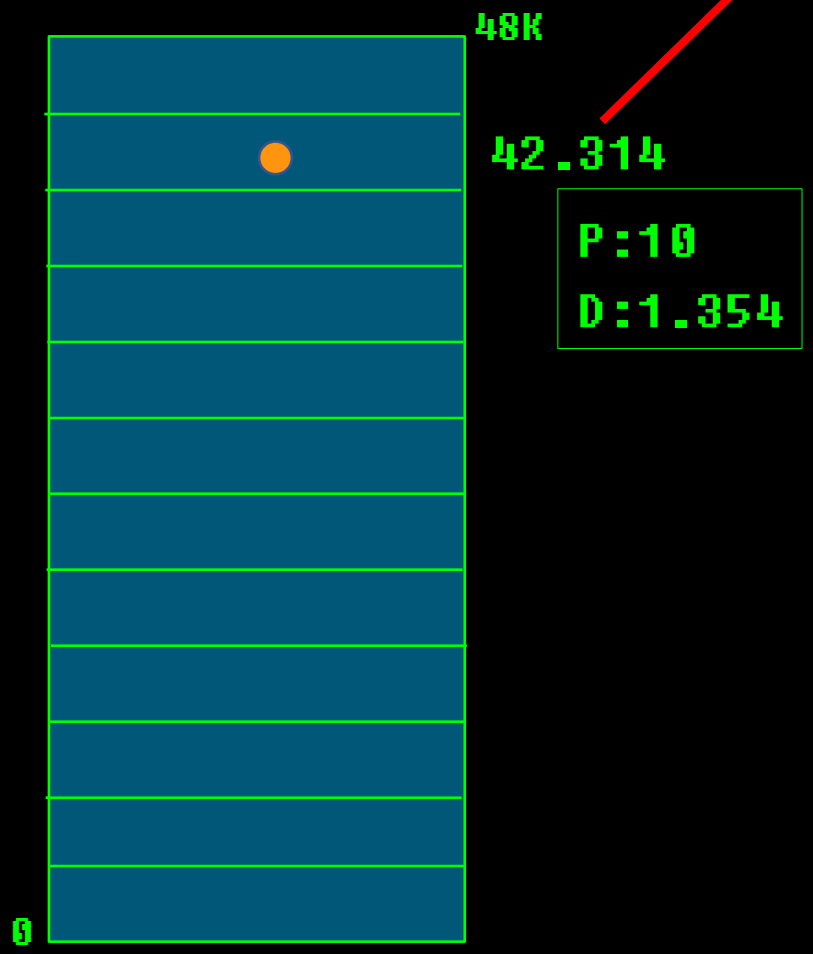
42.314

P:10
D:1.354

11	-
10	1
9	-
8	43
7	-
6	18
5	9
4	-
3	-
2	19
1	4
0	21



Paginação e memória virtual

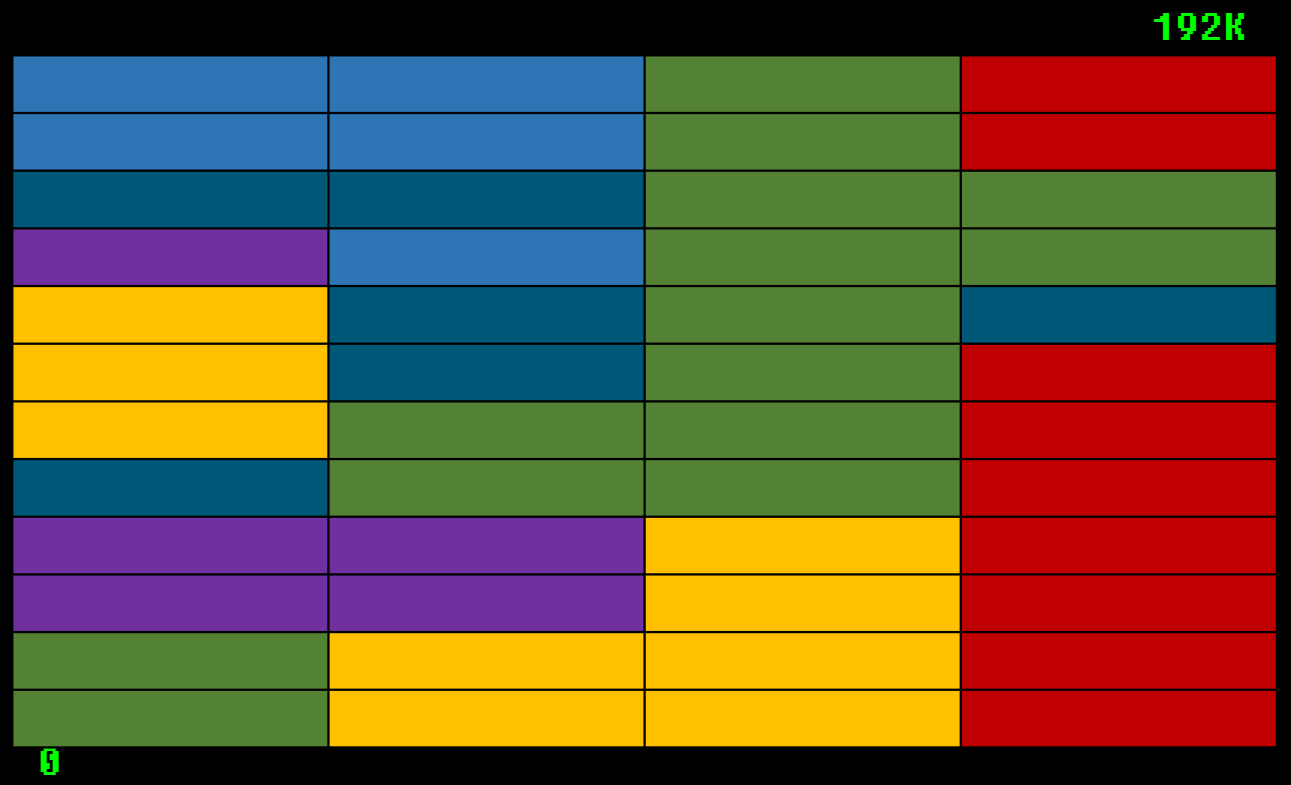


42.314

P:10
D:1.354

11	-
10	-
9	-
8	43
7	-
6	18
5	9
4	-
3	-
2	19
1	4
0	21

?



Memória virtual e substituição de páginas

- Se acontece uma falta de página e não há espaço na memória principal, é necessário *substituir* uma página

- Algoritmo ótimo: faz a melhor escolha possível para a retirada de uma página.
  - Qual é esta escolha?

Memória virtual e substituição de páginas

> Se acontece uma falta de página e não há espaço na memória principal, é necessário *substituir* uma página

> Algoritmo ótimo: faz a melhor escolha possível para a retirada de uma página.
  > Qual é esta escolha?

Memória virtual e substituição de páginas

- Se acontece uma falta de página e não há espaço na memória principal, é necessário *substituir* uma página

- Algoritmo ótimo: faz a melhor escolha possível para a retirada de uma página.
  - Qual é esta escolha?
  - Se não é possível esta escolha, o que fazer?

Algoritmos de substituição de páginas

Algoritmos de substituição

> Simulação de diferentes regras a partir de uma sequência de acessos de referência

> Comparação entre si e com o algoritmo ótimo

Algoritmo ótimo

> Memória com 4 molduras.

> Sequência: 3 2 1 0 3 4 3 1 1 5 6 1 0

Algoritmo FIFO

> A primeira página a entrar será a primeira a sair.

> Premissa: quem entrou primeiro deixará de ser importante mais cedo.

Algoritmo FIFO

> Memória com 4 molduras.

> Sequência: 3 2 1 0 3 4 3 1 1 5 6 1 0

Algoritmo NRU

- Página Não Recentemente Utilizada (*Not Recently Used*)
- SO usa dois bits de sinalização
  - R: referência (uso da página)
  - M: modificação (alteração do conteúdo da página)
- Periodicamente, o bit R é zerado
  - Conceito de “não recente”
  - Premissa do algoritmo
- Substituição por classe de prioridade

Classe	R	M
0	0	0
1	0	1
2	1	0
3	1	1

Algoritmo NRU

- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: 3 2* 1 0* 3 4* 3 1 1 5 6 1 0

Algoritmo de Segunda Chance

- Utiliza a lógica de FIFO.

- Mas a página com bit R = 1 ganha uma segunda chance
  - Zeramos o bit R e colocamos a página no fim da fila

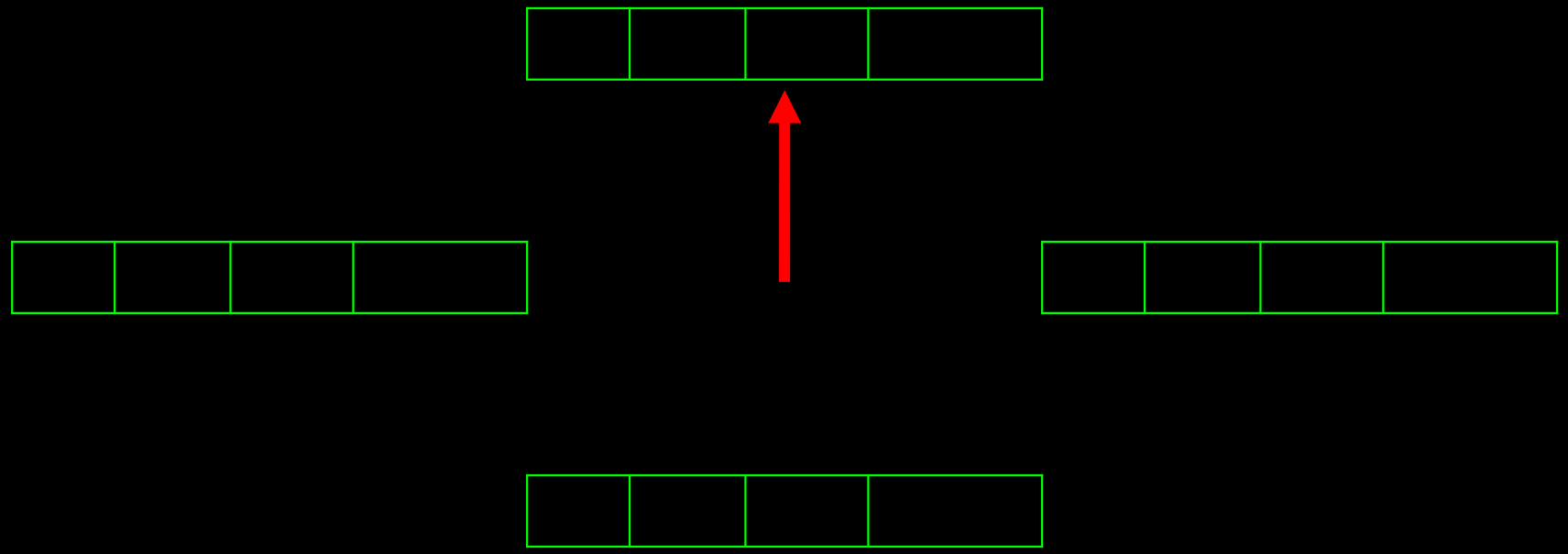
- Premissa: quem entrou primeiro deixará de ser importante mais cedo, *a não ser que continue sendo utilizada.*

Algoritmo de Segunda Chance / Relógio

- Segunda chance é geralmente implementada com uma lista circular, por questão de desempenho
  - Algoritmo do *relógio*

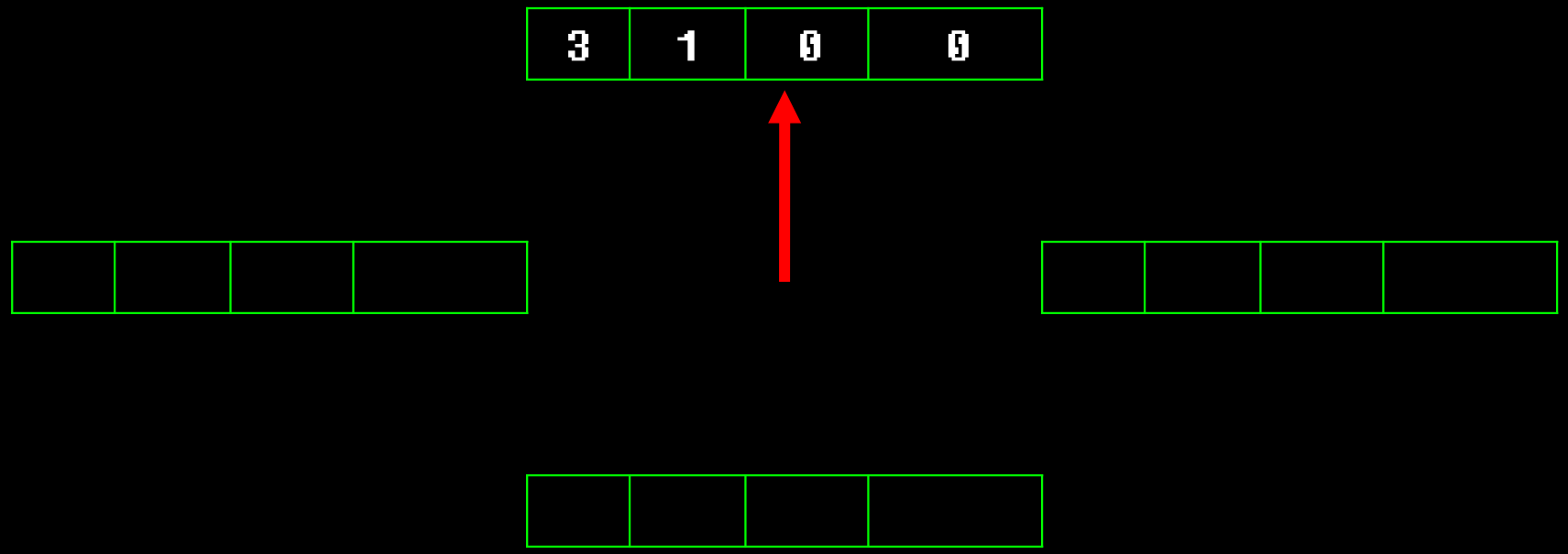
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: 3 2* 1 0* 3 4* 3 1 1 5 6 1 0



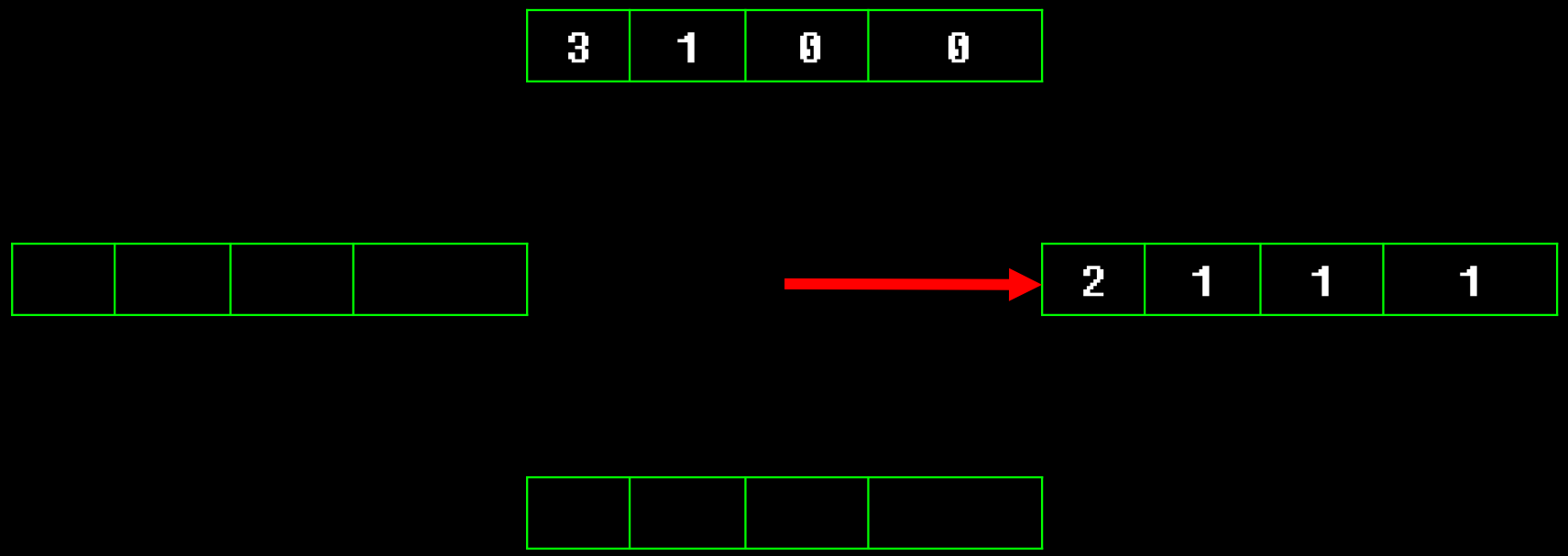
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: 3 2* 1 0* 3 4* 3 1 1 5 6 1 0



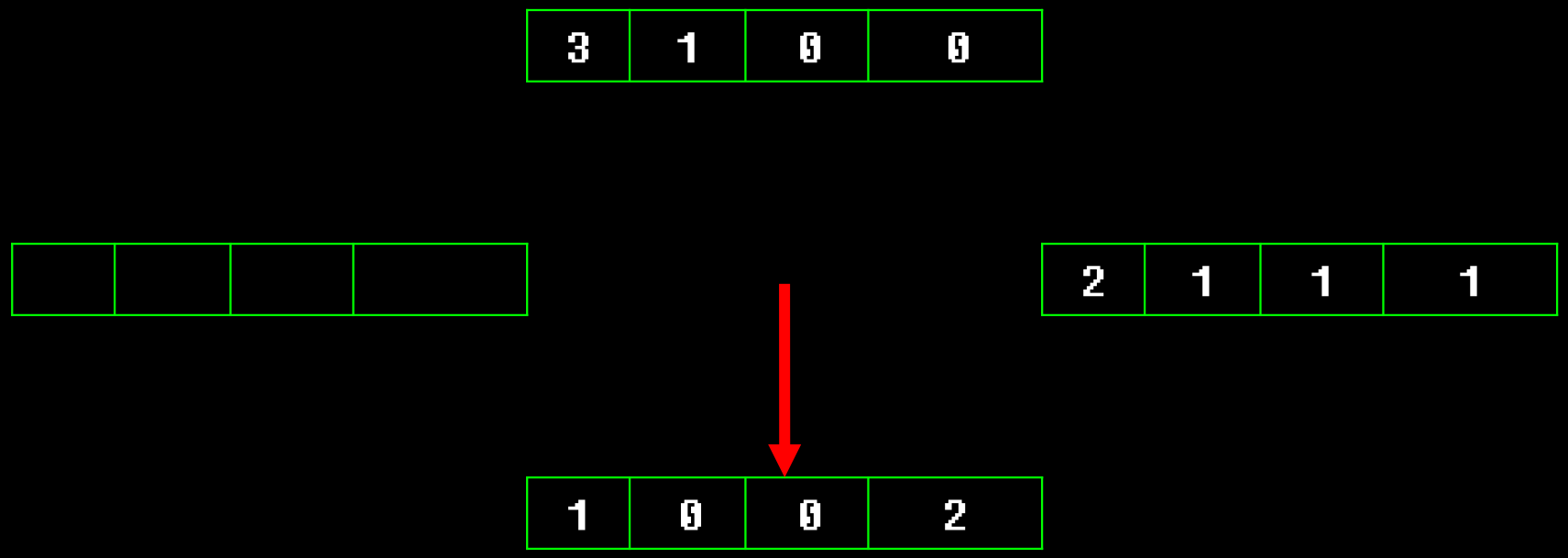
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: 3 2* 1 0* 3 4* 3 1 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: 3 2* 1 0* 3 4* 3 1 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: **3 2* 1 0* 3 4* 3 1 1 5 6 1 0**

3	1	0	0

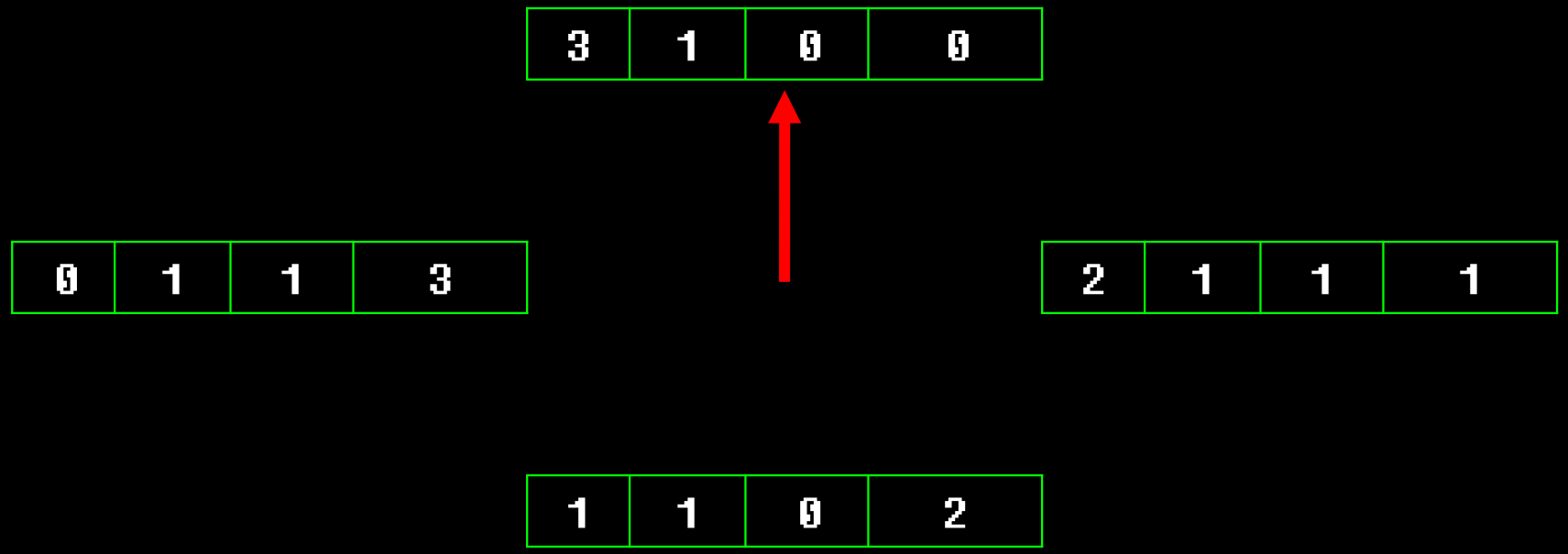
0	1	1	3

2	1	1	1

1	1	0	2

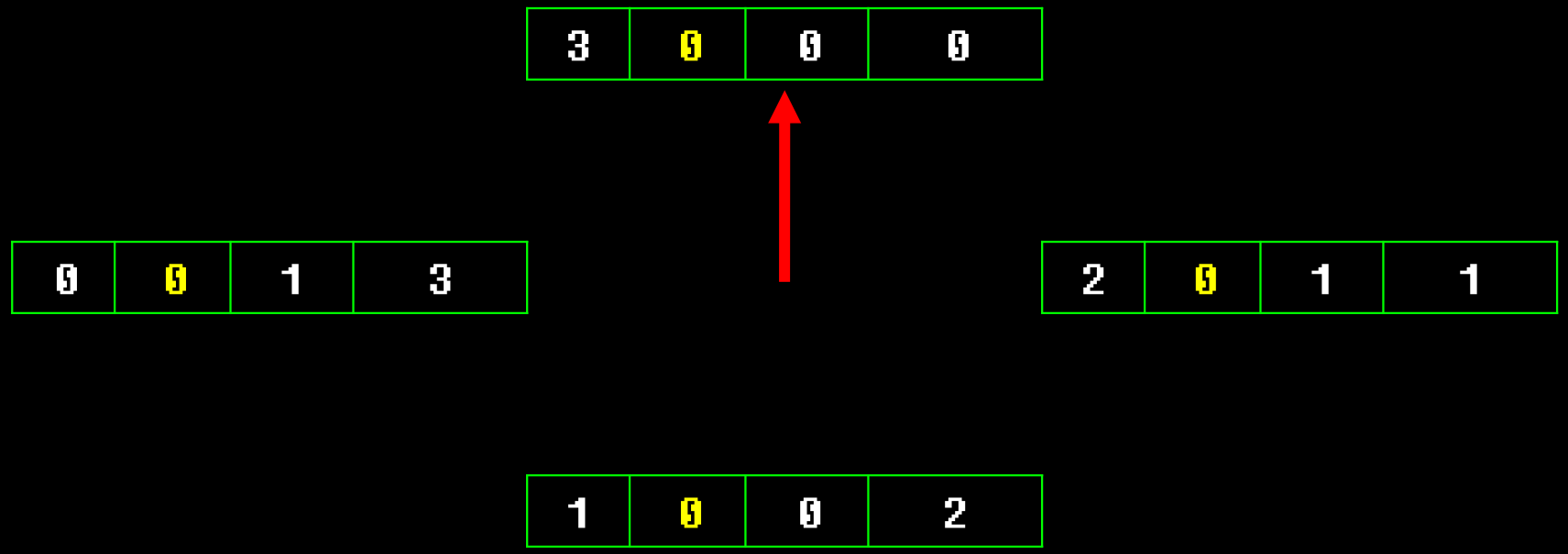
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: 3 2* 1 0* 3 4* 3 1 1 5 6 1 0



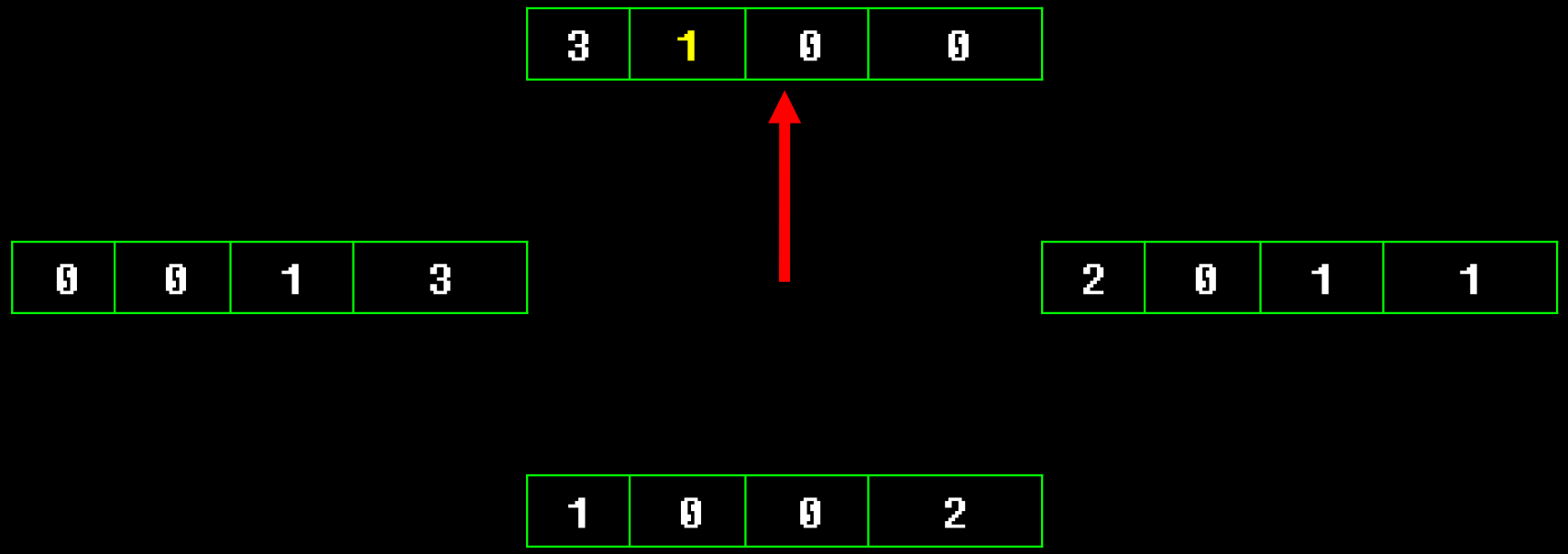
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0*~~ 3 4* 3 1 1 5 6 1 0



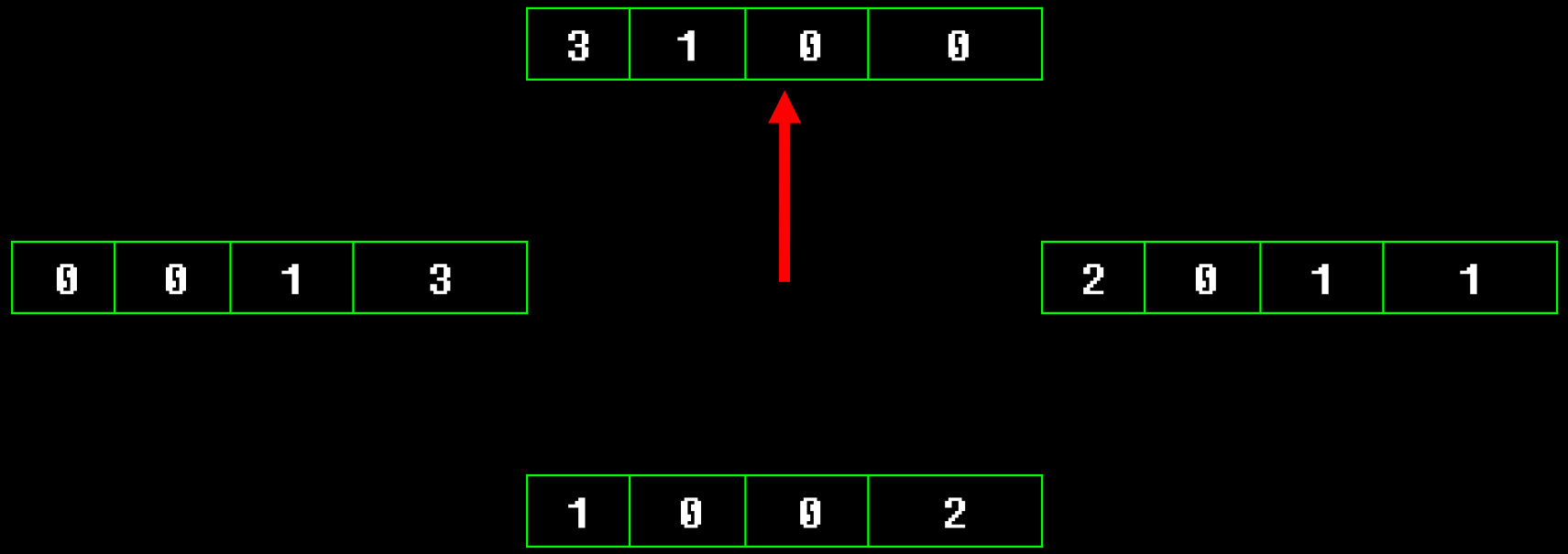
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0*~~ 3 4* 3 1 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0*~~ 3 4* 3 1 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0*~~ 3 4* 3 1 1 5 6 1 0

3	0	0	0

0	0	1	3

2	0	1	1

1	0	0	2

Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0*~~ 3 4* 3 1 1 5 6 1 0

3	0	0	0

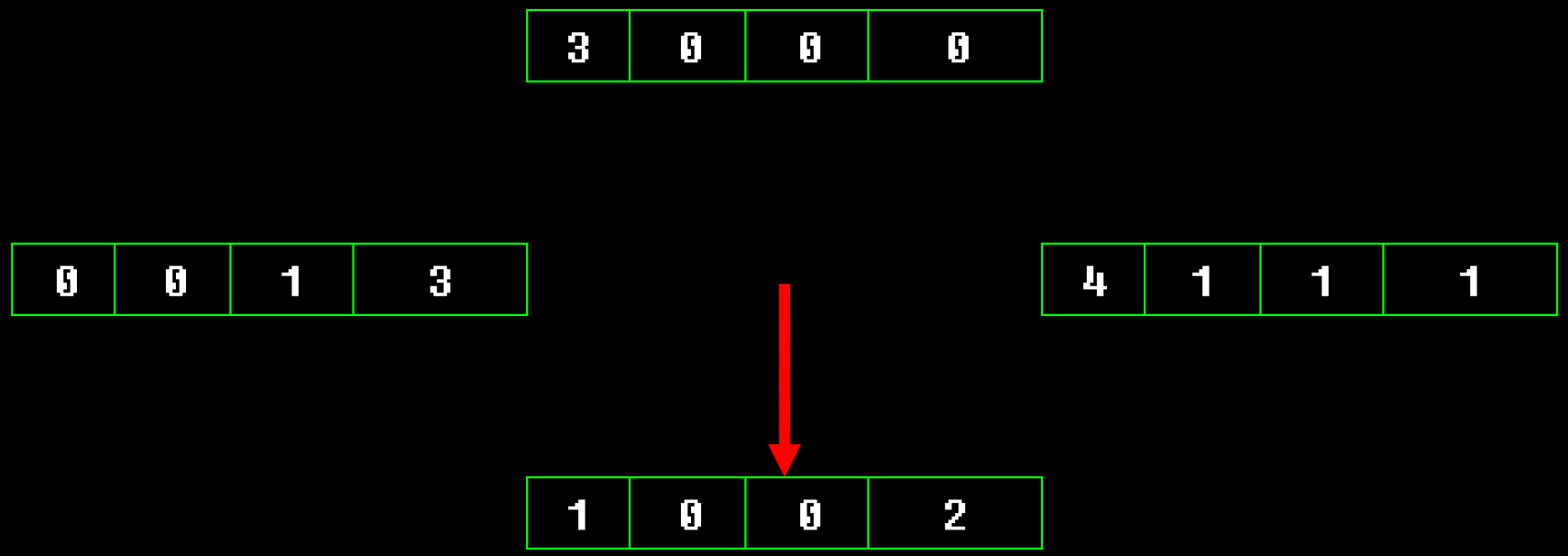
0	0	1	3

4	1	1	1

1	0	0	2

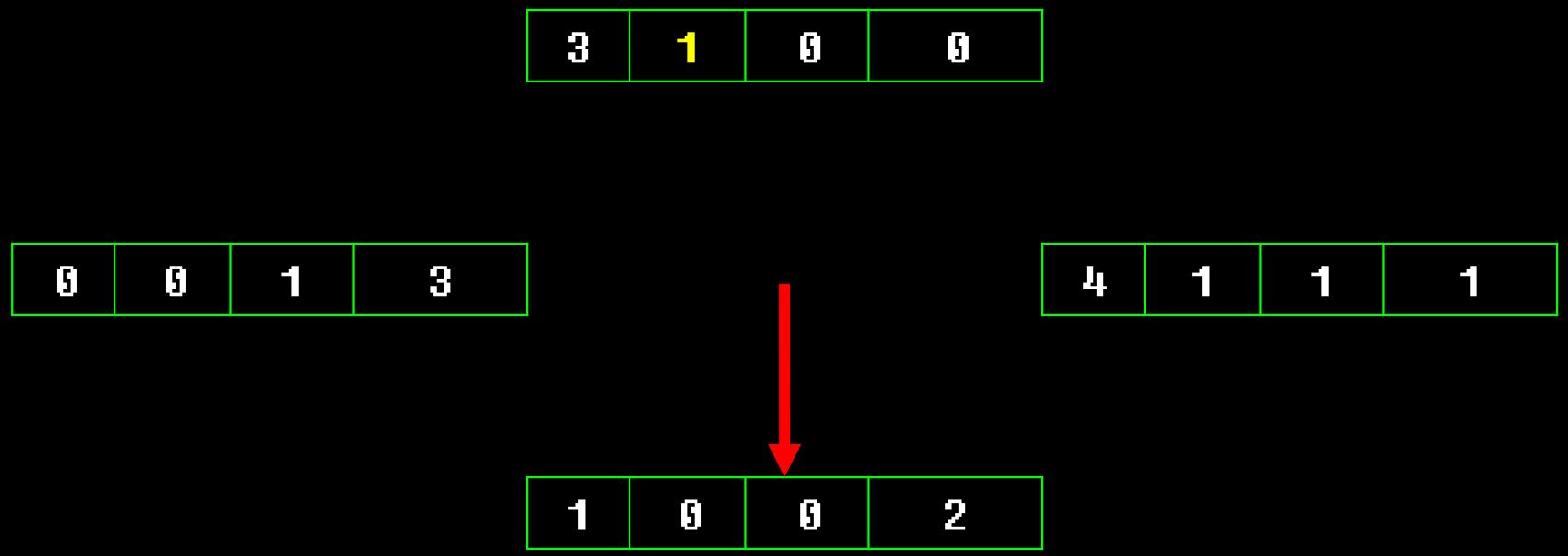
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0*~~ 3 4* 3 1 1 5 6 1 0



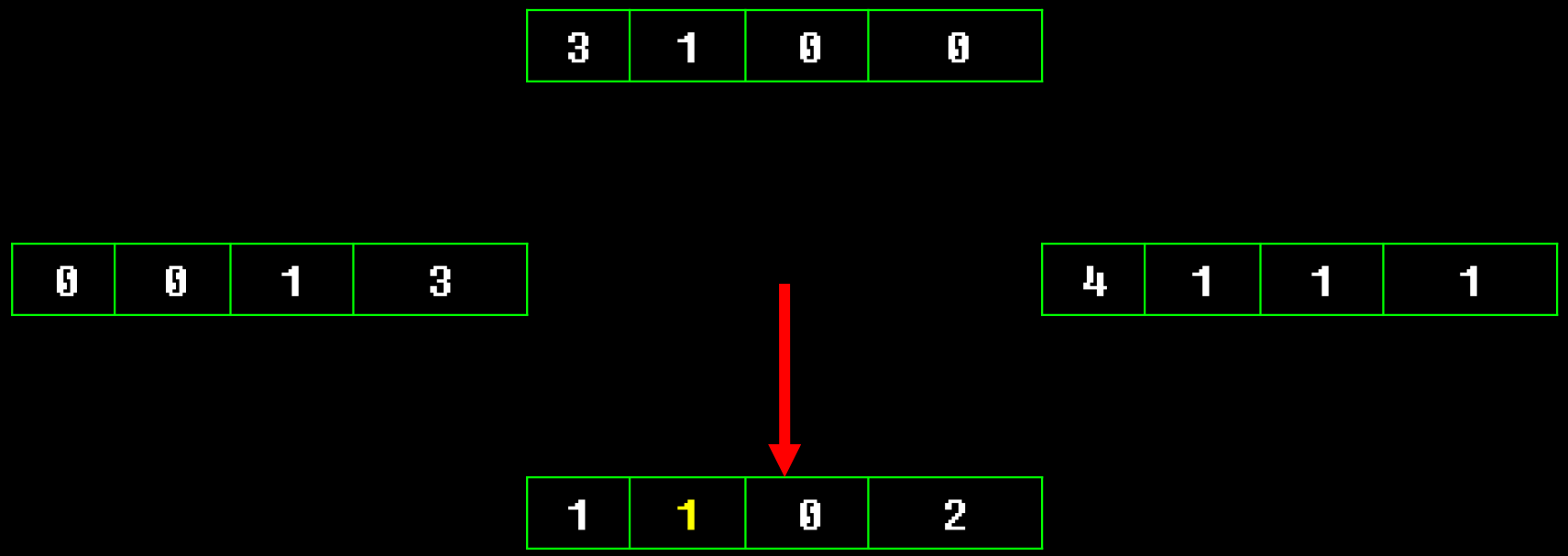
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0*~~ 3 4* 3 1 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0*~~ 3 4* 3 1 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0

3	0	0	0

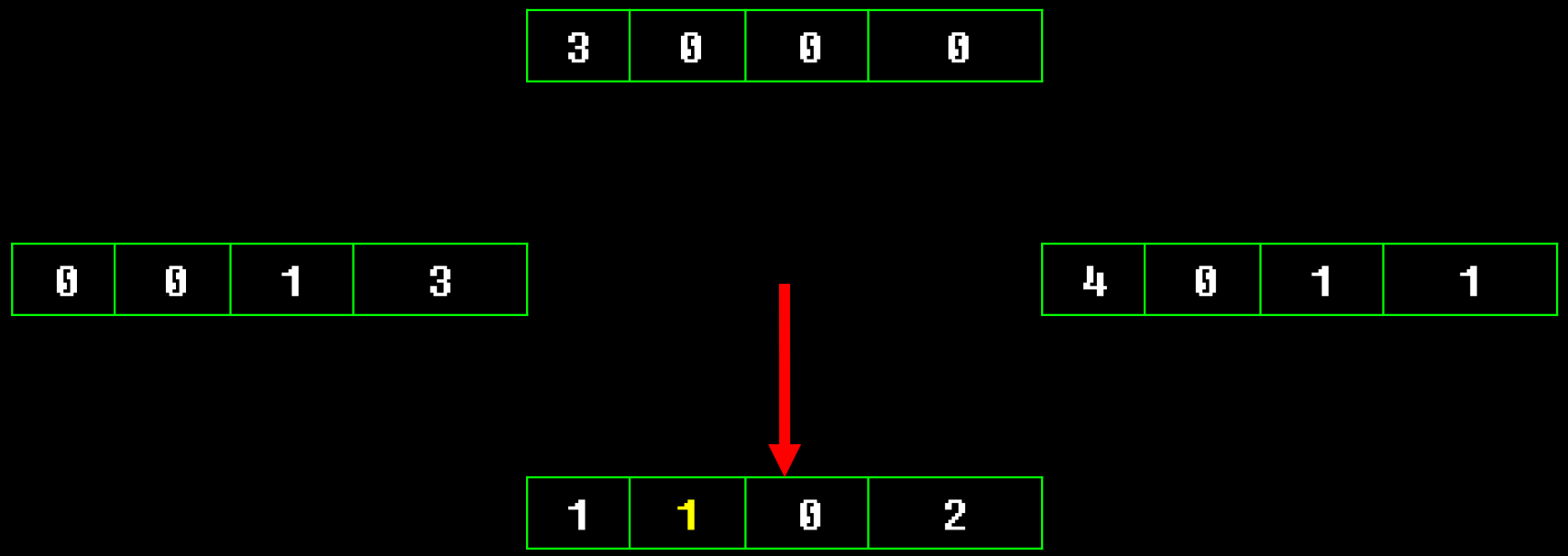
0	0	1	3

4	0	1	1

1	0	0	2

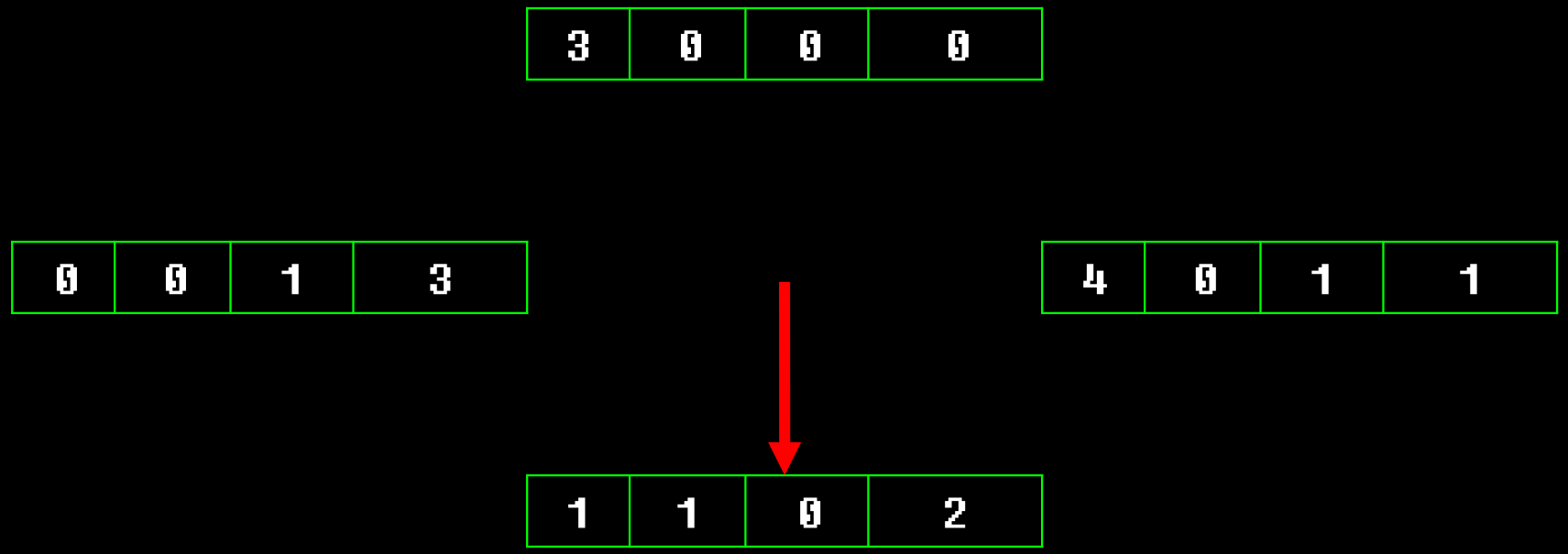
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0

3	0	0	0

0	0	1	3

4	0	1	1

1	0	0	2

Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0

3	0	0	0

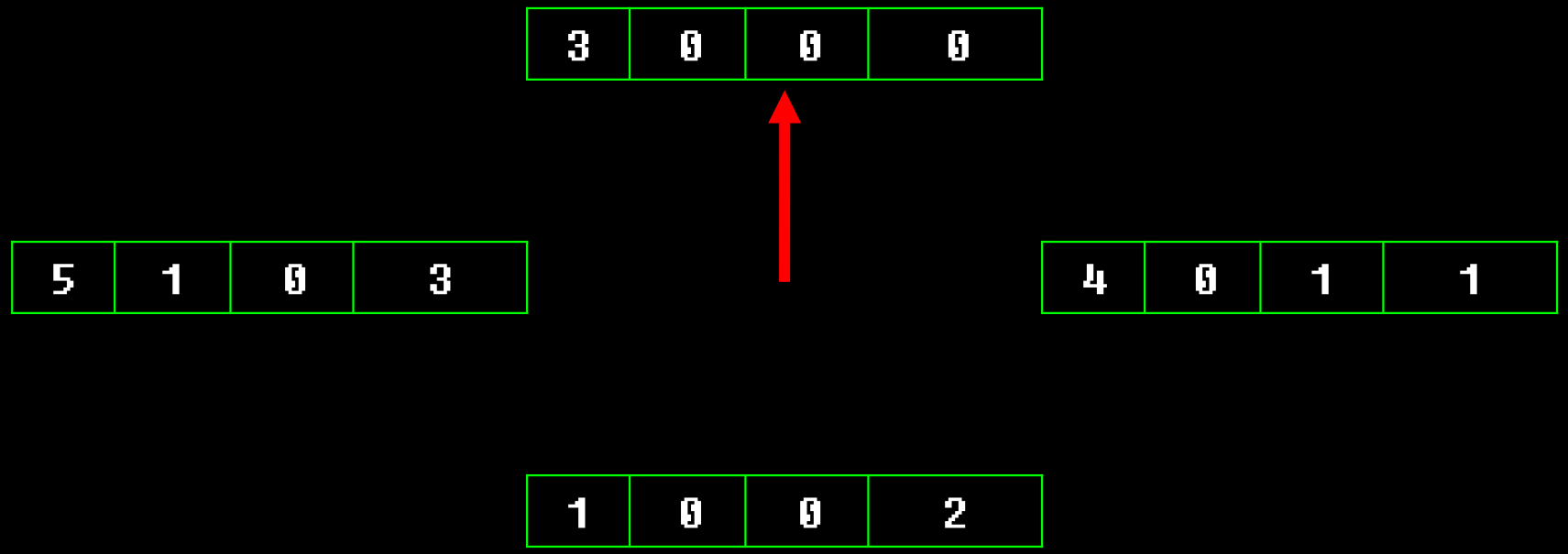
5	1	0	3

4	0	1	1

1	0	0	2

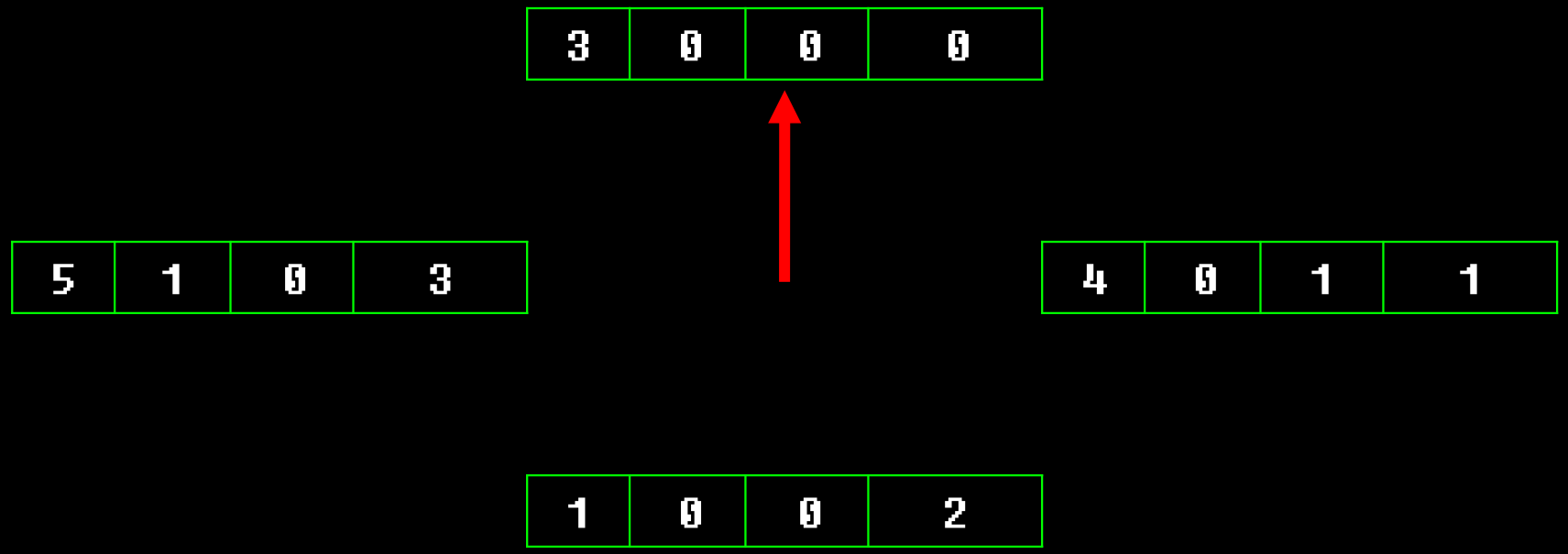
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0



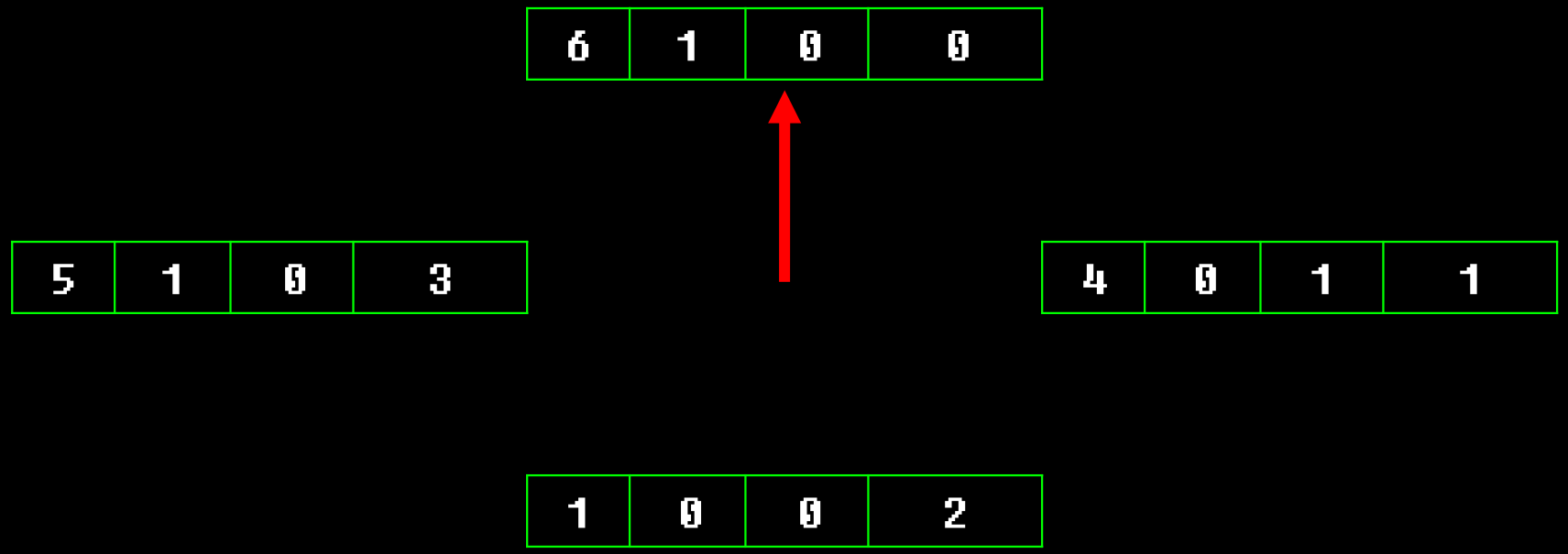
Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0



Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0

6	1	0	0

5	1	0	3

4	0	1	1

1	0	0	2

Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1~~ 1 5 6 1 0

6	1	0	0

5	1	0	3

4	0	1	1

1	1	0	2

Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1 1 5 6 1~~ 0

6	0	0	0

5	0	0	3

4	0	1	1

1	0	0	2

Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1 1 5 6 1~~ 0

6	0	0	0

5	0	0	3

4	0	1	1

1	0	0	2

Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1 1 5 6 1~~ 0

6	0	0	0

5	0	0	3

0	1	0	1

1	0	0	2

Algoritmo de Segunda Chance / Relógio

- Algoritmo do *relógio*
- Memória com 4 molduras. Limpeza a cada 4 acessos
  - Marca de modificação: *
- Sequência: ~~3 2* 1 0* 3 4* 3 1 1 5 6 1~~ 0

6	0	0	0

5	0	0	3

0	1	0	1

1	0	0	2

Algoritmo LFU

- Página menos frequentemente utilizada (*L*east *F*requently *U*sed)

- Contador de acessos a cada página: sai a que tiver menor contador

- Premissa: uma página pouco usada até o momento provavelmente será pouco importante no futuro.

Algoritmo LFU

- Página menos frequentemente utilizada (*L*east *F*requently *U*sed)
- Contador de acessos a cada página: sai a que tiver menor contador
- É barato manter o contador?
- E se a página acabou de entrar, qual será seu contador?

Algoritmo LFU com envelhecimento

- Simulação do contador real: conta *intervalos* de uso

- Envelhecimento: tentativa de “esquecer” valores de acessos antigos
  - A página é importante se continuar sendo utilizada

- Uso do bit R para implementação

Algoritmo LFU com envelhecimento

- Uso do bit R para implementação

- Conjunto de bits de referência: “idade”
  - Bit R é o bit à esquerda da “idade”

- A cada época, *shift* à direita da idade
  - *Envelhecimento*

- Sai a página com menor valor de “idade” (menos uso, ou usos mais antigos)

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: 3 2 1 0 3 4 3 1 1 5 6 1 0

Página	Idade

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: 3 2 1 0 3 4 3 1 1 5 6 1 0

Página	Idade
3	1000

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: 3 2 1 0 3 4 3 1 1 5 6 1 0

Página	Idade
3	1000
2	1000

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: 3 2 1 0 3 4 3 1 1 5 6 1 0

Página	Idade
3	1000
2	1000
1	1000

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: 3 2 1 0 3 4 3 1 1 5 6 1 0

Página	Idade
3	1000
2	1000
1	1000
0	1000

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0~~ 3 4 3 1 1 5 6 1 0

Página	Idade
3	0100
2	0100
1	0100
0	0100

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0~~ 3 4 3 1 1 5 6 1 0

Página	Idade
3	1100
2	0100
1	0100
0	0100

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0~~ 3 4 3 1 1 5 6 1 0

Página	Idade
3	1100
2	0100
1	0100
0	0100

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0~~ 3 4 3 1 1 5 6 1 0

Página	Idade
3	1100
4	1000
1	0100
0	0100

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0~~ 3 4 3 1 1 5 6 1 0

Página	Idade
3	1100
4	1000
1	0100
0	0100

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0~~ 3 4 3 1 1 5 6 1 0

Página	Idade
3	1100
4	1000
1	1100
0	0100

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1~~ 1 5 6 1 0

Página	Idade
3	0110
4	0100
1	0110
0	0010

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1~~ 1 5 6 1 0

Página	Idade
3	0110
4	0100
1	1110
0	0010

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1~~ 1 5 6 1 0

Página	Idade
3	0110
4	0100
1	1110
0	0010

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1~~ 1 5 6 1 0

Página	Idade
3	0110
4	0100
1	1110
5	1000

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1~~ 1 5 6 1 0

Página	Idade
3	0110
4	0100
1	1110
5	1000

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1~~ 1 5 6 1 0

Página	Idade
3	0110
6	1000
1	1110
5	1000

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1~~ 1 5 6 1 0

Página	Idade
3	0110
6	1000
1	1110
5	1000

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1 1 5 6 1~~ 0

Página	Idade
3	0011
6	0100
1	0111
5	0100

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1 1 5 6 1~~ 0

Página	Idade
3	0011
6	0100
1	0111
5	0100

Algoritmo LFU com envelhecimento

> Memória com 4 molduras. Idade de 4 bits.

> Limpeza a cada 4 acessos

> Sequência: ~~3 2 1 0 3 4 3 1 1 5 6 1~~ 0

Página	Idade
0	1000
6	0100
1	0111
5	0100

Algoritmos globais x algoritmos locais

- Globais: escolhem uma página de qualquer processo em memória principal para substituir

- Locais: escolhem uma página do próprio processo em memória principal para substituir

Conjunto de trabalho (*working set*)

- Conjunto mínimo de páginas necessário para o processo realizar o seu trabalho

- Definição do conjunto de trabalho:
  - Previsão do futuro
  - Estimativa a partir de um intervalo de tempo passado

- Proteção de uma página do conjunto
  - Bit L (*lock*)

Algoritmo WSClock

- Usa o conceito de conjunto de trabalho e o método de implementação do relógio.

- Regras:
  - Se R==1, zerar o bit R e seguir em frente
  - Se R==0, verificar idade (tempo - intervalo)
    - Se idade válida, seguir em frente
    - Se idade inválida, verificar bit M
      - Se M==0, substituir página
      - Se M==1, marcar página e procurar outra candidata

Algoritmo WSClock

Tempo atual: 2204
Reset de R: 100
Intervalo WS: 200

4	0	1	1620

6	0	1	2084

5	1	1	2132

8	0	0	2003

3	1	0	2120

2	0	0	1213

7	0	0	2014

9	0	1	1980

Algoritmo WSClock

Tempo atual: 2204
Reset de R: 100
Intervalo WS: 200

4	0	1	1620

6	0	1	2084

5	1	1	2132

8	0	0	2003

3	0	0	2120

2	0	0	1213

7	0	0	2014

9	0	1	1980

Algoritmo WSClock

Tempo atual: 2204
Reset de R: 100
Intervalo WS: 200

4	0	1	1620

6	0	1	2084

5	1	1	2132

8	0	0	2003

3	0	0	2120

2	0	0	1213

7	0	0	2014

9	0	1	1980

Algoritmo WSClock

Tempo atual: 2204
Reset de R: 100
Intervalo WS: 200

4	0	1	1620

6	0	1	2084

5	1	1	2132

8	0	0	2003

3	0	0	2120

2	0	0	1213

7	0	0	2014

9	0	1	1980

Algoritmo WSClock

Tempo atual: 2204
Reset de R: 100
Intervalo WS: 200

4	0	1	1620

6	0	1	2084

5	1	1	2132

8	0	0	2003

3	0	0	2120

2	0	0	1213

7	0	0	2014

9	0	1	1980

Algoritmo WSClock

Tempo atual: 2204
Reset de R: 100
Intervalo WS: 200

4	0	1	1620

6	0	1	2084

5	1	1	2132

8	0	0	2003

3	0	0	2120

2	0	0	1213

7	0	0	2014

9	0	1	1980

Algoritmo WSClock

Tempo atual: 2204
Reset de R: 100
Intervalo WS: 200

4	0	1	1620

6	0	1	2084

5	1	1	2132

8	0	0	2003

3	0	0	2120

10	1	0	2204

7	0	0	2014

9	0	1	1980

Algoritmo WSClock

Tempo atual: 2205
Reset de R: 100
Intervalo WS: 200

4	0	1	1620

6	0	1	2084

5	1	1	2132

8	0	0	2003

3	0	0	2120

10	1	0	2204

7	0	0	2014

9	0	1	1980

Memória Virtual

Uma técnica

Introdução

- memória principal semicondutora
  - capacidade limitada
  - tempo de acesso entre 10 e 20 ns
- memória secundária em disco
  - Capacidade limitada (obviamente) mas muito maior
  - tempo de latência entre 10 e 30 ms

O problema

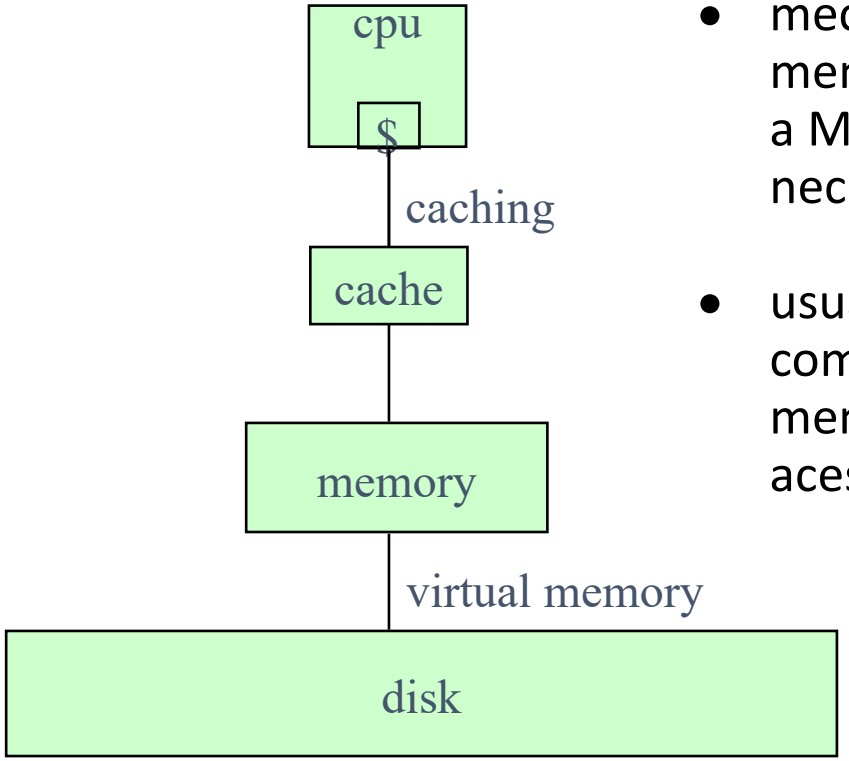
- Um computador tem 32 kbytes de memória principal

- Como podemos:
  - rodar programas que usam mais do que 32 kbytes?
  - permitir que vários usuários usem o computador?
  - executar vários programas ao mesmo tempo?

- O problema é independente do tamanho da memória. Pode trocar para 32 MB ou 32 GB, que o problema permanece. A memória é dimensionada para atender a carga de trabalho típica do sistema.

Memória virtual: a solução

- *Memória Virtual* : técnica que nos permite ver a memória principal como uma cache de grande capacidade de armazenamento
- É apenas mais um nível na hierarquia de memórias



- mecanismo automático de gerência de memória, que traz automaticamente para a MP os blocos de informação (do disco) necessários
- usuário tem a impressão de trabalhar com uma memória única, do tamanho da memória secundária, mas com tempo de acesso próximo do tempo da MP

Tempo de acesso

Tempo médio de acesso Tma é dado por

$$Tma = Tm + (1 – h) Ts$$

onde Tm = tempo de acesso à MP
Ts = tempo de acesso ao disco
h = hit ratio

p.ex. se Tm = 20 ns, Ts = 20 ms, h = 0.9999
então Tma = 2,02 μs (100 x maior do que Tm)

Por que MV é diferente das caches?

- Miss penalty é MUITO maior (milhões de ciclos)! Se informação não está na memória, está no disco!

- Logo:
  - miss ratio precisa ser bem menor do que em cache
  - alta penalidade do miss => necessário buscar blocos maiores em disco
  - princípio de localidade opera sobre blocos maiores de dados ou instruções e leva a hit ratios bem mais elevados
  - Mapeamento totalmente associativo das páginas
  - misses são tratados por software (há tempo disponivel)
  - técnica de escrita write-through não é uma opção. Usa-se write-back.

Terminologia

- mesma idéia da cache, mas com terminologia diferente

cache	MV
bloco	página
cache miss	*page fault*
endereço	endereço virtual (ou lógico)
índice	endereço real (ou físico)

- endereço *virtual (lógico)*: gerado pelo programa
  - deve endereçar todo espaço em disco
  - maior número de bits

- endereço *real (físico)*: endereço na memória principal
  - menor número de bits

Unidade de Gerenciamento de Memória

- MMU (Memory Management Unit)
  - gerência da hierarquia de memória
  - proteção de memória
  - usualmente integrada dentro do microprocessador
- MMU deve fazer mapeamento do endereço virtual para endereço real
- SO usa a MMU

Paginação (gerenciamento de memória)

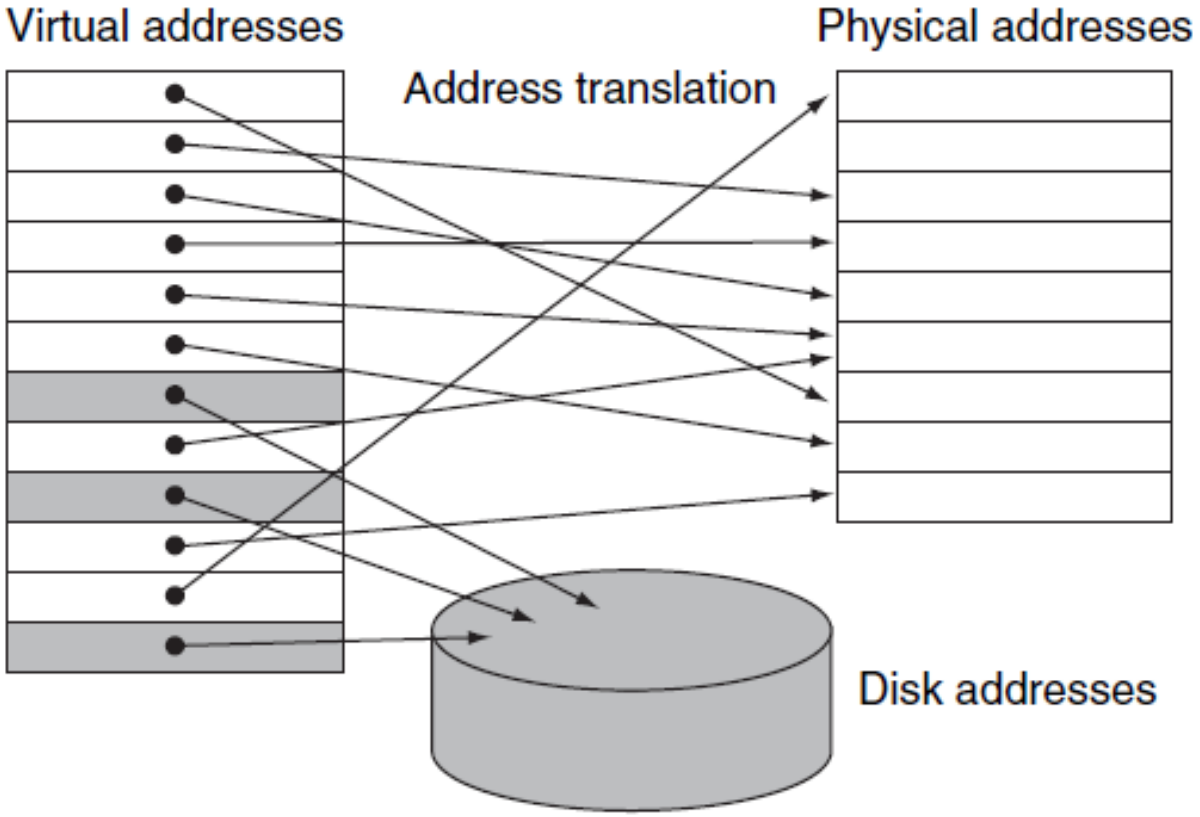
- Por que paginação? Resposta: mecanismo simples para tradução de endereços virtuais em reais e para gerenciamento do espaço de memória

- espaços de memória real e virtual divididos em blocos chamados de páginas
  - páginas tem tipicamente de 4 kbytes a 16 kbytes
  - Páginas para sistemas embarcados são de 1 kbytes

- endereços virtuais e reais divididos em 2 campos
  - endereço da página
  - endereço da linha (ou palavra), dentro da página

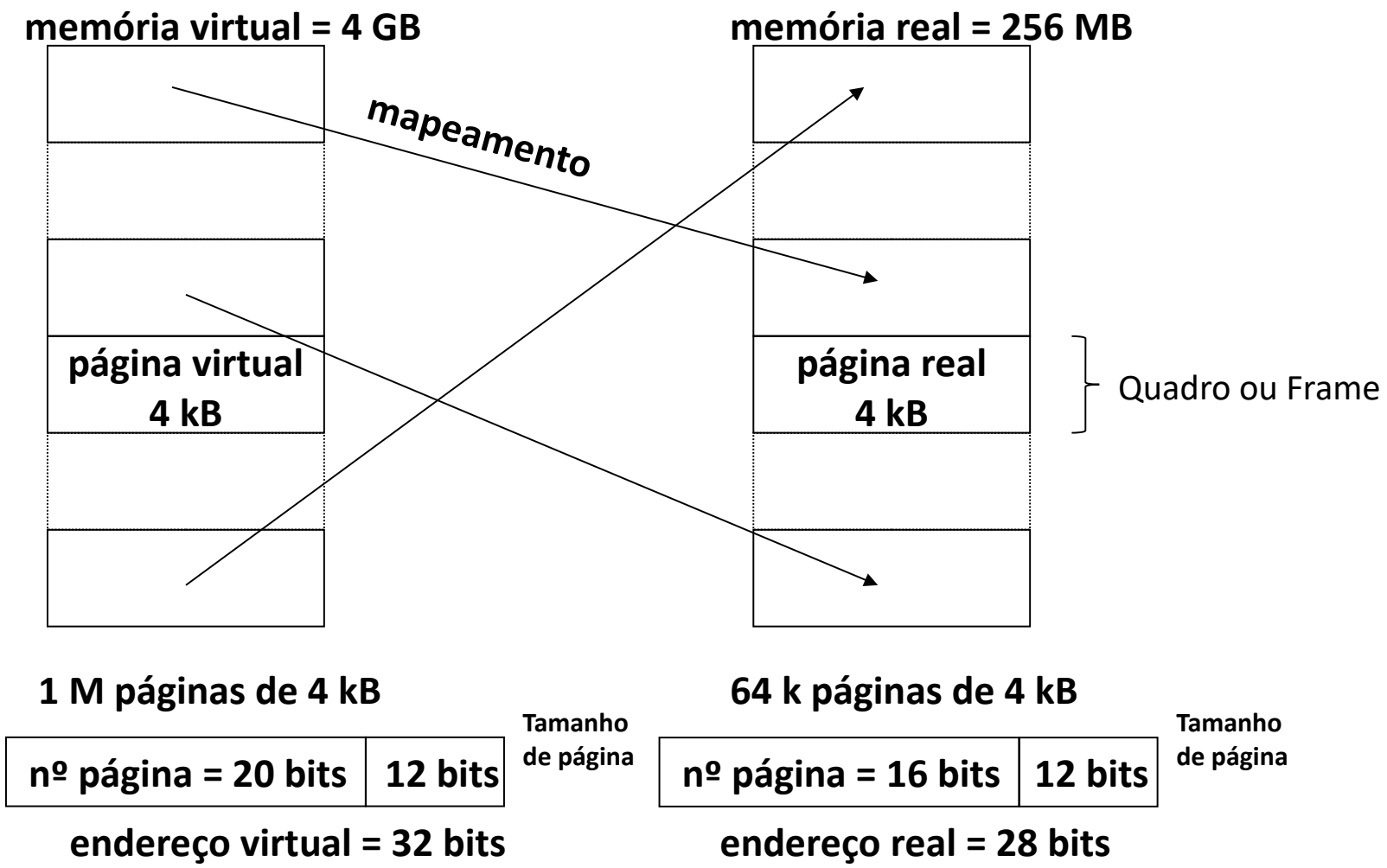
Paginação

- É apenas uma função de mapeamento dos endereços virtuais (do disco) para endereços reais (físicos) na memória principal



PATTERSON, David A. Organização e projeto de computadores a interface hardware/software

Paginação


memória virtual = 4 GB
memória real = 256 MB
mapeamento
página virtual
4 kB
página real
4 kB
Quadro ou Frame
1 M páginas de 4 kB
nº página = 20 bits
12 bits
Tamanho
de página
endereço virtual = 32 bits
64 k páginas de 4 kB
nº página = 16 bits
12 bits
Tamanho
de página
endereço real = 28 bits

Paginação

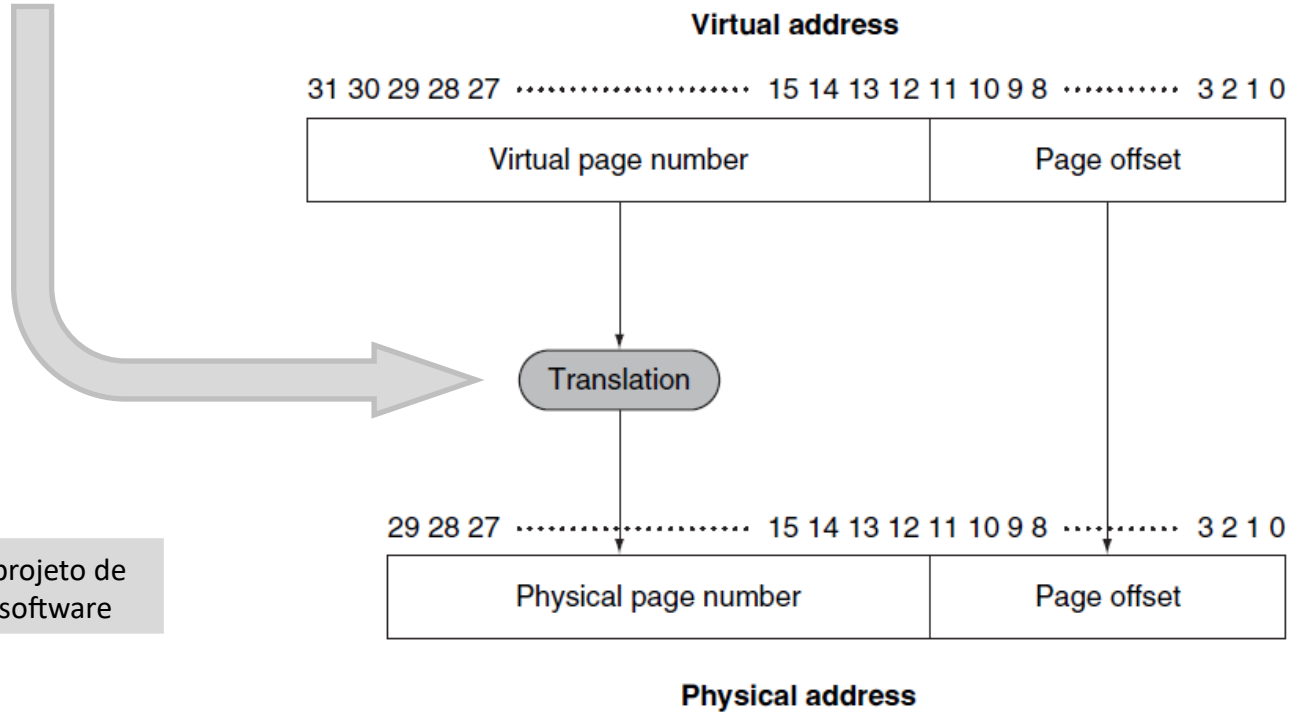
- Page fault ocorre quando a página virtual não está na memória principal
- Mapeamento completamente associativo, mais eficiente, ajuda a diminuir alta penalidade dos page faults

- Como transformar endereçamento original do programa no endereçamento real?

- page tables
  - guardam a correspondência entre páginas virtuais e páginas reais
  - permitem a translação de endereços

Paginação

- Como transformar endereçamento original do programa no endereçamento real?
- Page tables
  - guardam a correspondência entre páginas virtuais e páginas reais
  - permitem a translação de endereços



PATTERSON, David A. Organização e projeto de computadores a interface hardware/software

Gerência de processos

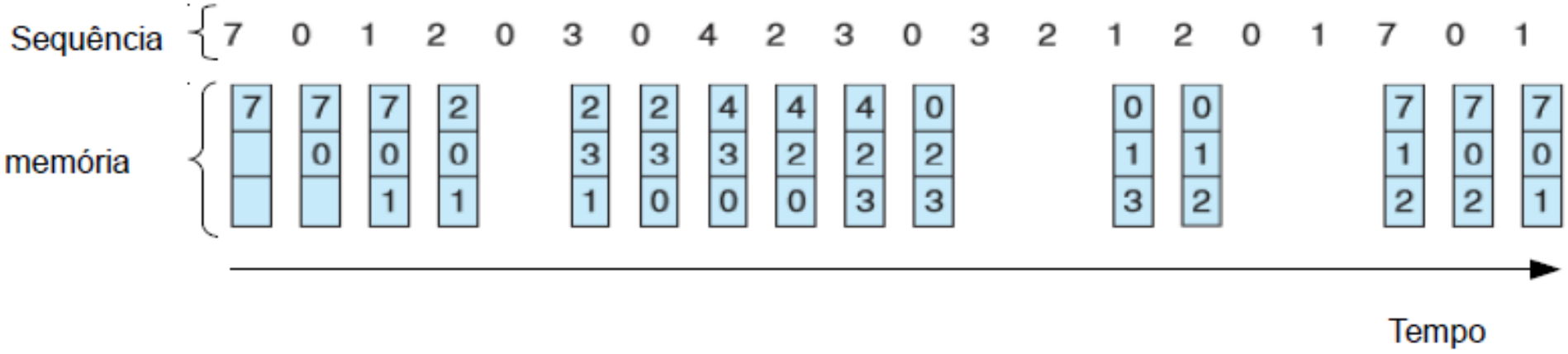
- cada processo tem sua própria tabela de páginas
  - processos são compilados para espaços de endereçamento virtuais
  - tabela de páginas define toda a utilização do espaço de endereçamento pelo processo
- sistema operacional é responsável pela alocação de espaço físico para o espaço virtual de cada processo
  - SO carrega tabela de páginas de cada processo
- hardware possui registrador que aponta para início da tabela de páginas do processo atual
- quando novo processo passa a ser ativo, sistema operacional só precisa atualizar valor deste registrador

Paginação

- main memory translation table (MMTT)
  - implementada em hardware (*em tese*)
  - tamanho = nº de páginas na memória principal
- disk memory translation table (DMTT)
  - implementada em software, armazenada na memória principal
  - tamanho = nº de páginas em disco
- **algoritmo de substituição**, em software, para selecionar página da memória principal a ser substituída em caso de page fault
- bom desempenho é garantido pelo princípio de localidade

Substituição de páginas

- FIFO
- Memória com 3 frames (quadros)
  - Frame tem tamanho igual a uma página virtual

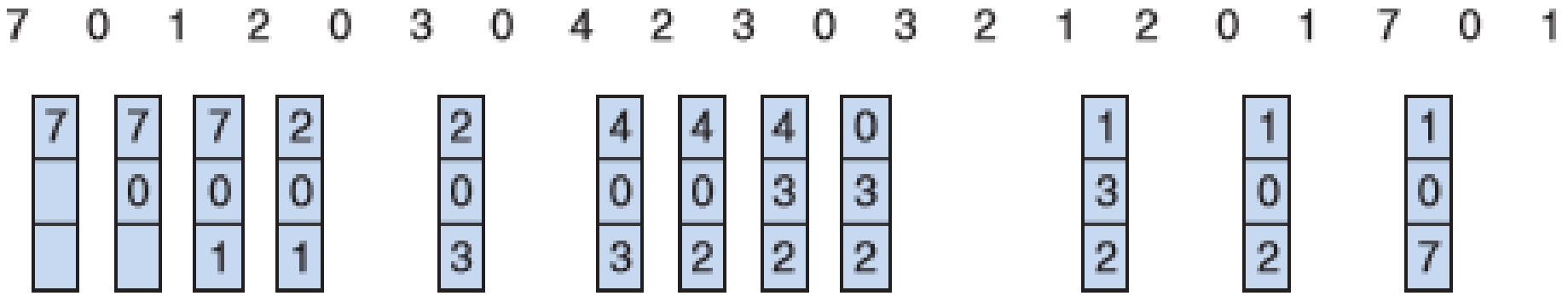


  - Total de falhas = 15
  - Taxa de falha = 15/20 = 0,75

https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/9_VirtualMemory.html

Substituição de páginas

- LRU
- Memória com 3 frames (quadros)
  - Frame tem tamanho igual a uma página virtual

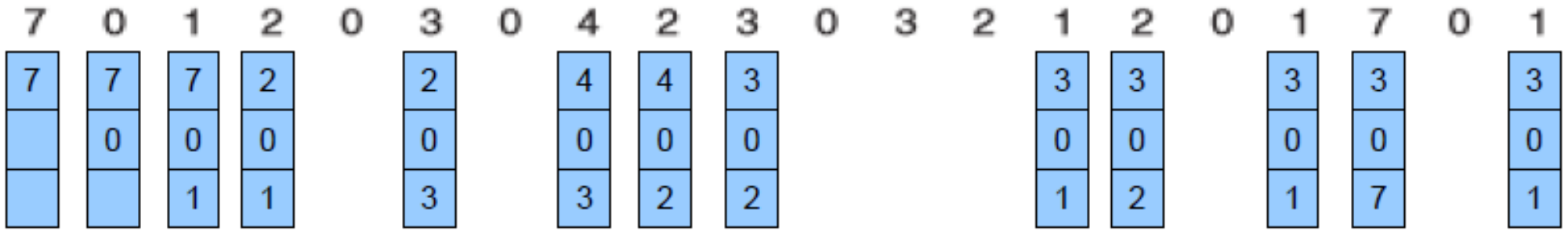


  - Total de requisições = 20
  - Total de falhas = 12
  - Taxa de falha = 12/20 = 0,6

https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/9_VirtualMemory.html

Substituição de páginas

- LFU
  - FIFO como critério de desempate
- Memória com 3 frames (quadros)
  - Frame tem tamanho igual a uma página virtual

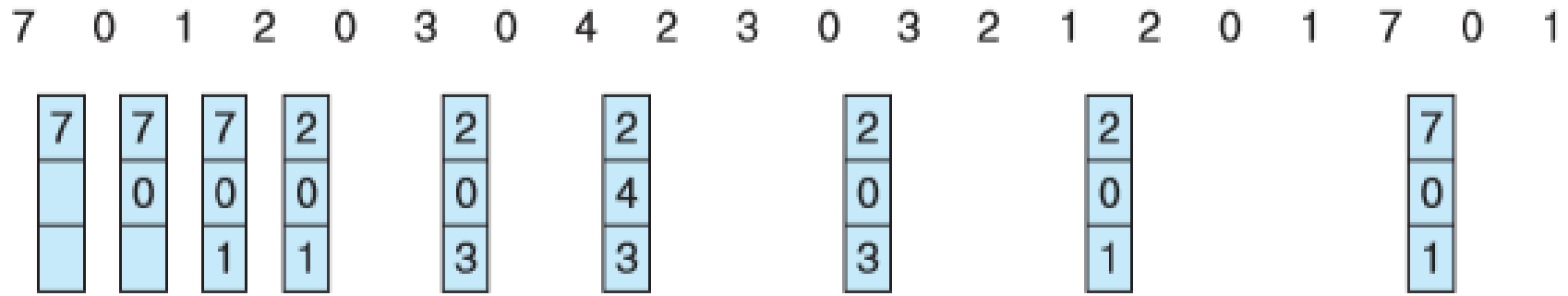


  - Total de requisições = 20
  - Total de falhas = 13
  - Taxa de falha = 13/20 = 0,65

https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/9_VirtualMemory.html

Substituição de páginas

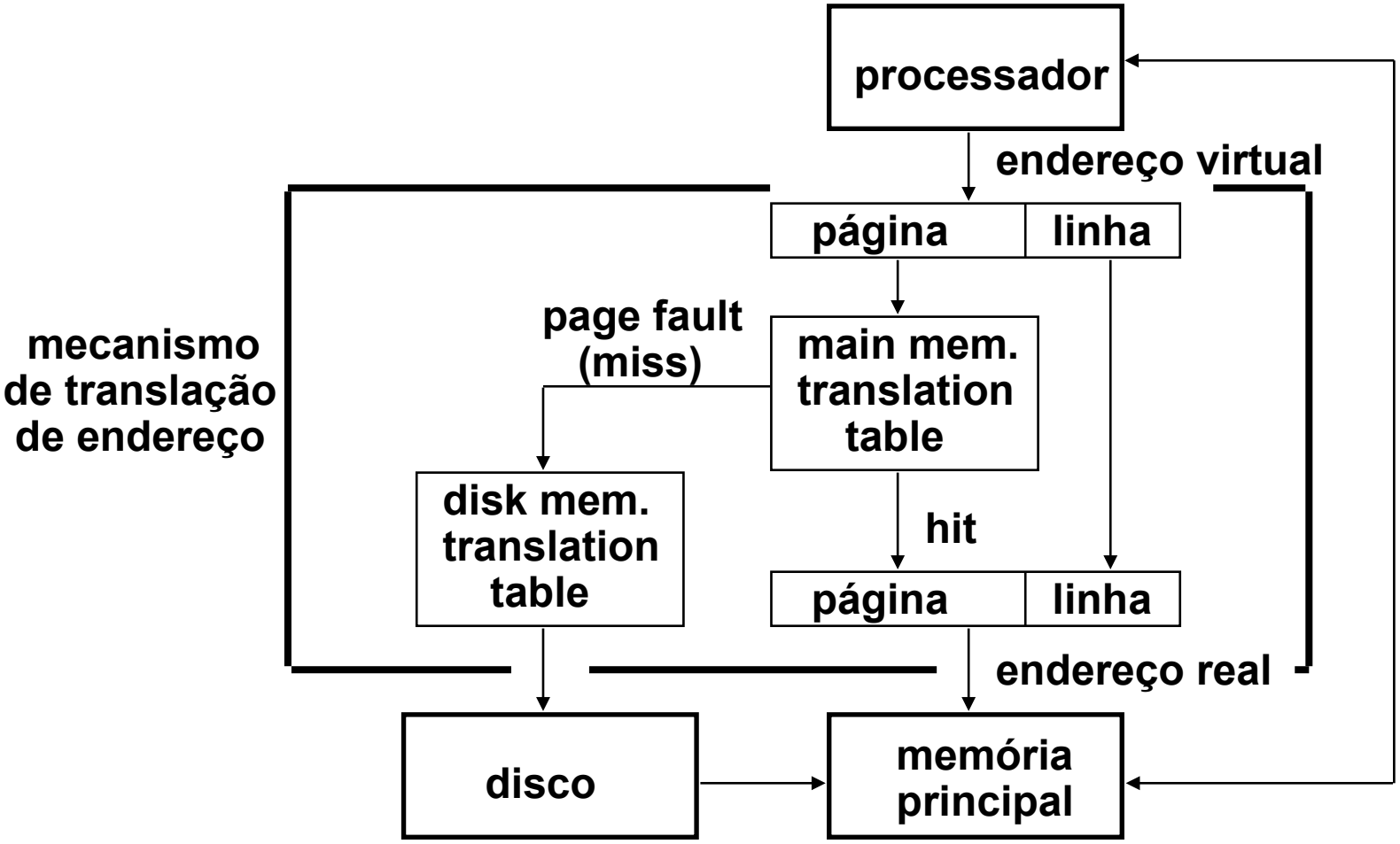
- OPT (solução ótima)
  - Substitui a página que não será usada por um tempo mais longo (prever futuro)
- Memória com 3 frames (quadros)
  - Frame tem tamanho igual a uma página virtual



  - Total de requisições = 20
  - Total de falhas = 9
  - Taxa de falha = 9/20 = 0,45

https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/9_VirtualMemory.html

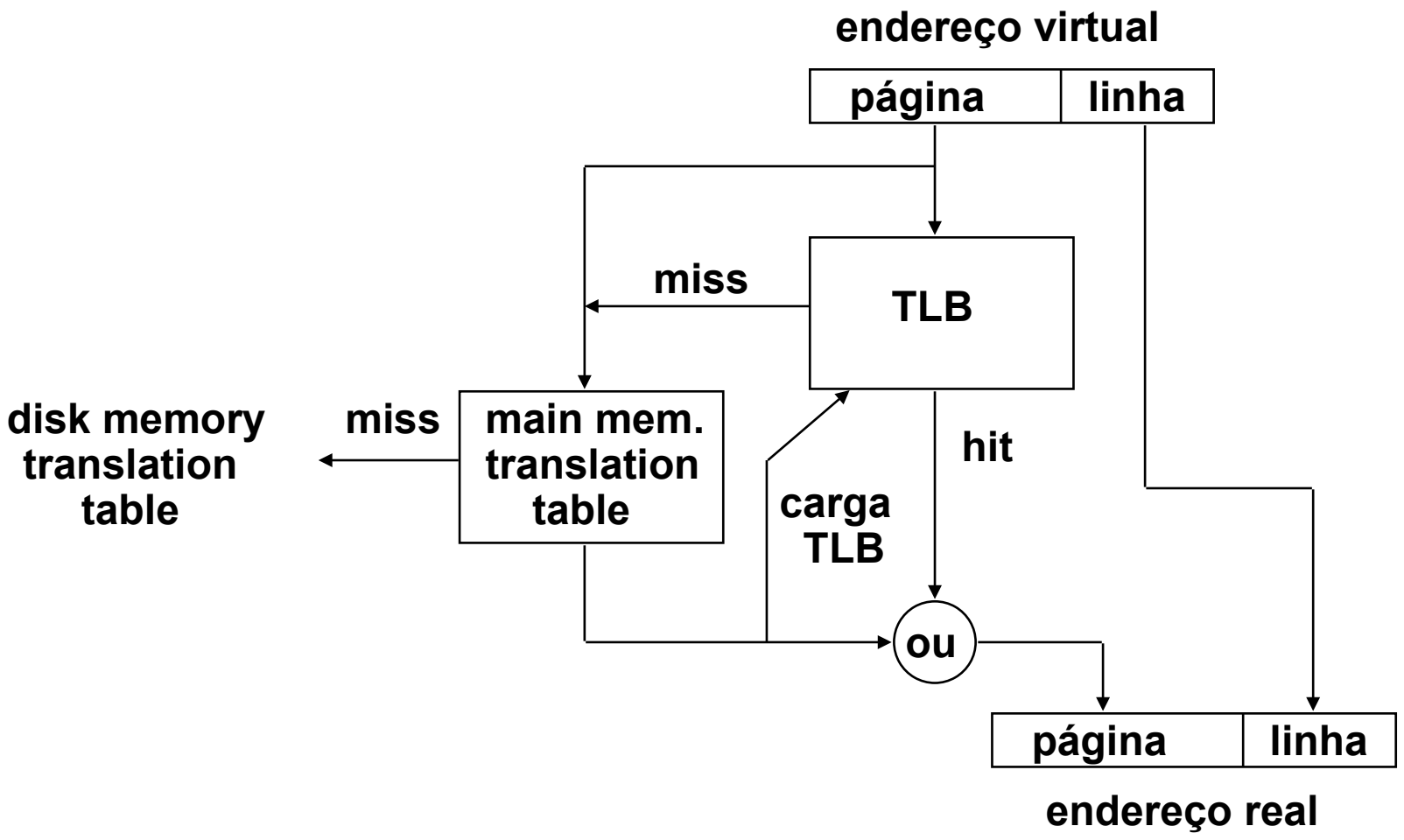
Paginação (translação do endereço)



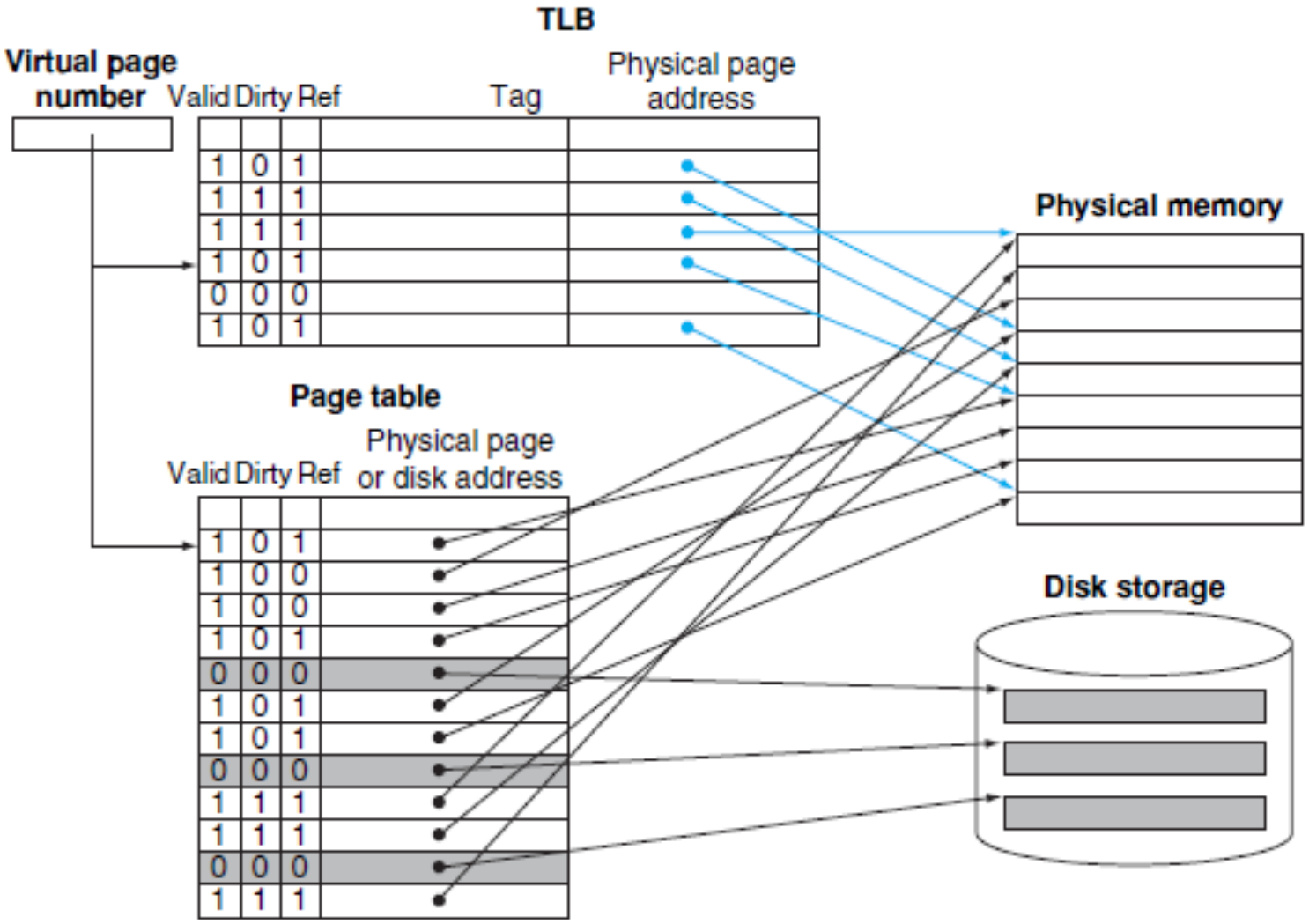
Translation Look-Aside Buffer

- nº de páginas na memória secundária é muito grande
  - espaço virtual de 2^{32}bytes, páginas de 4k bytes, 4 bytes por entrada na tabela
  - 4 MBytes apenas para a tabela de páginas!!!
  - tamanho excessivo da main memory translation table
- se tabela ficar na memória principal => dois acessos à memória a cada cache miss
- working set = conjunto de páginas mais prováveis de serem acessadas num dado momento, devido ao princípio de localidade
- Translation Look-Aside Buffer (TLB)
  - implementado em hardware
  - traduz endereços virtuais para endereços reais
  - só inclui páginas do working set
  - pode ser considerado como uma “cache” da MMTT
- Main Memory Translation Table (MMTT)
  - implementada em **software**

Translation Look-Aside Buffer

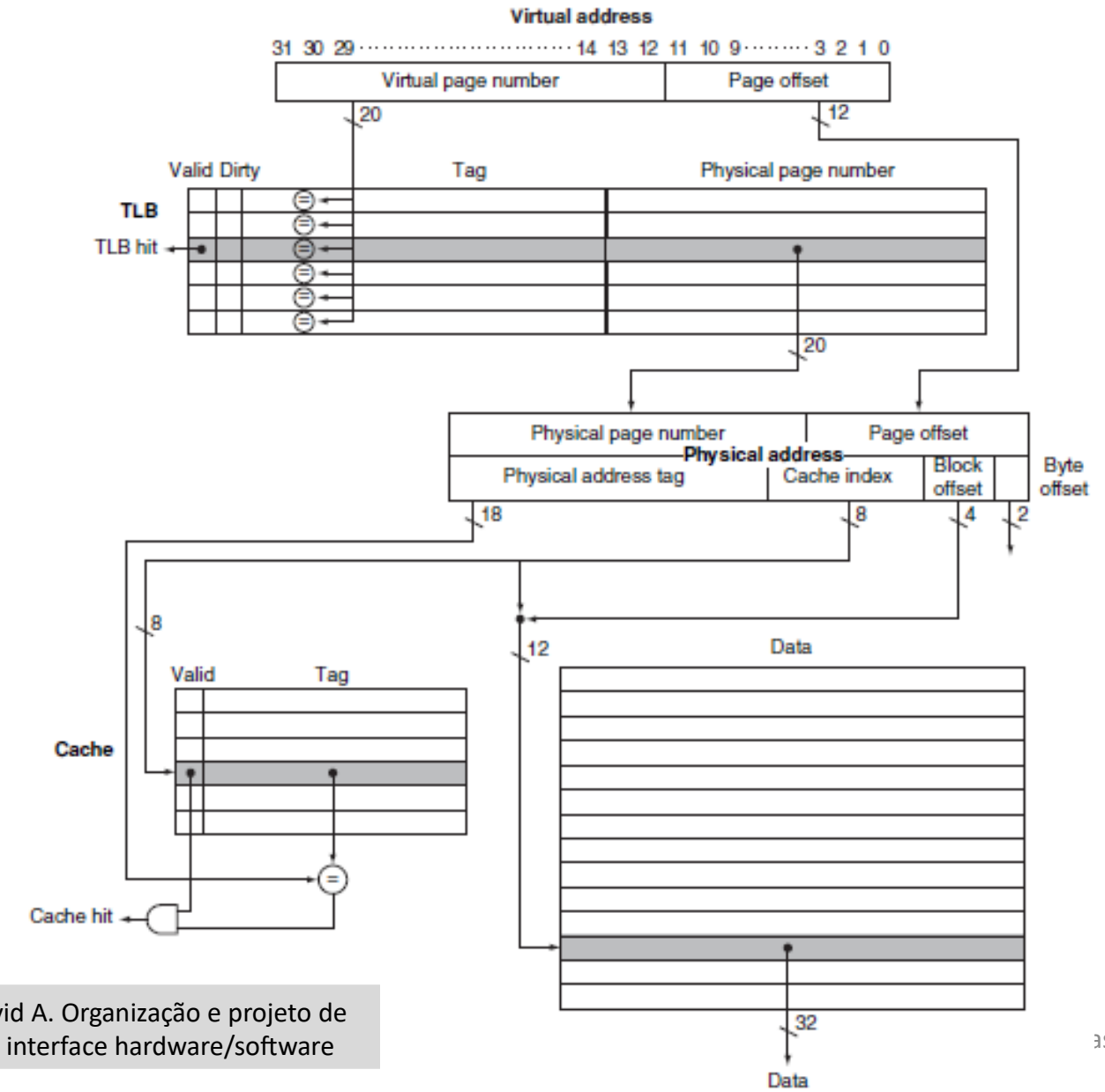


Translation Look-Aside Buffer



PATTERSON, David A. Organização e projeto de computadores a interface hardware/software

Translation Look-Aside Buffer



Cache física ou
Cache virtual?

Qual a diferença?

PATTERSON, David A. Organização e projeto de computadores a interface hardware/software

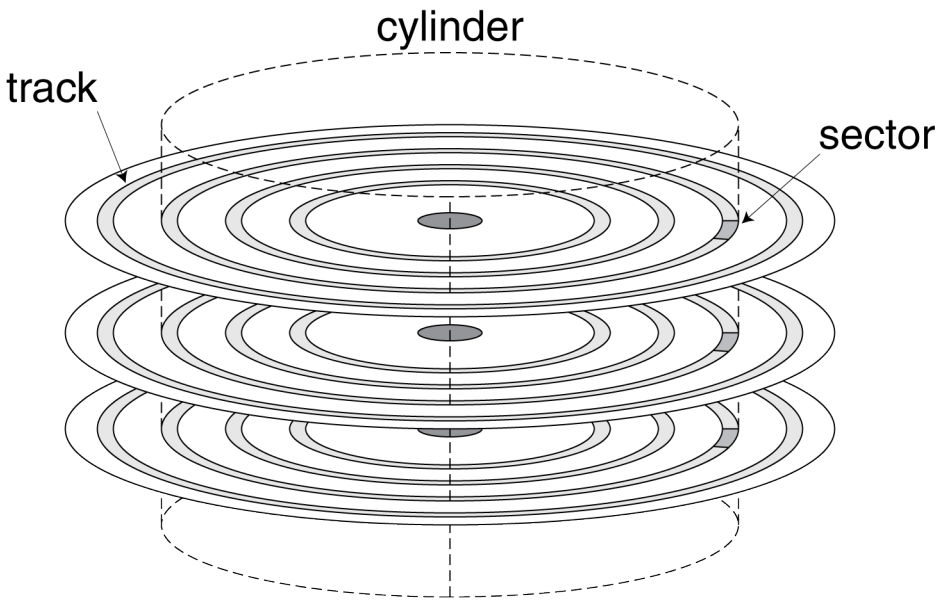
Translation Look-Aside Buffer

TLB	Page table	Cache	Possible? If so, under what circumstance?
Hit	Hit	Miss	Possible, although the page table is never really checked if TLB hits.
Miss	Hit	Hit	TLB misses, but entry found in page table; after retry, data is found in cache.
Miss	Hit	Miss	TLB misses, but entry found in page table; after retry, data misses in cache.
Miss	Miss	Miss	TLB misses and is followed by a page fault; after retry, data must miss in cache.
Hit	Miss	Miss	Impossible: cannot have a translation in TLB if page is not present in memory.
Hit	Miss	Hit	Impossible: cannot have a translation in TLB if page is not present in memory.
Miss	Miss	Hit	Impossible: data cannot be allowed in cache if the page is not in memory.

PATTERSON, David A. Organização e projeto de computadores a interface hardware/software

Disco

- Para acessar os dados:
  - Tempo de busca: posiciona a cabeça sobre a trilha correta (média de 3 a 14 ms)
  - Latência rotacional: espera pelo setor desejado (0,5 rpm)
  - Tempo de transferência: recupera os dados (um ou ou mais setores; ex: 30 a 80 MB/seg)



PATTERSON, David A. Organização e projeto de computadores a interface hardware/software

Desempenho do Disco

- Tempo de Disco = Tempo de busca + Latência rotacional + Tempo de transferência

- A latência rotacional média para a informação desejada está a meio caminho ao redor do disco.

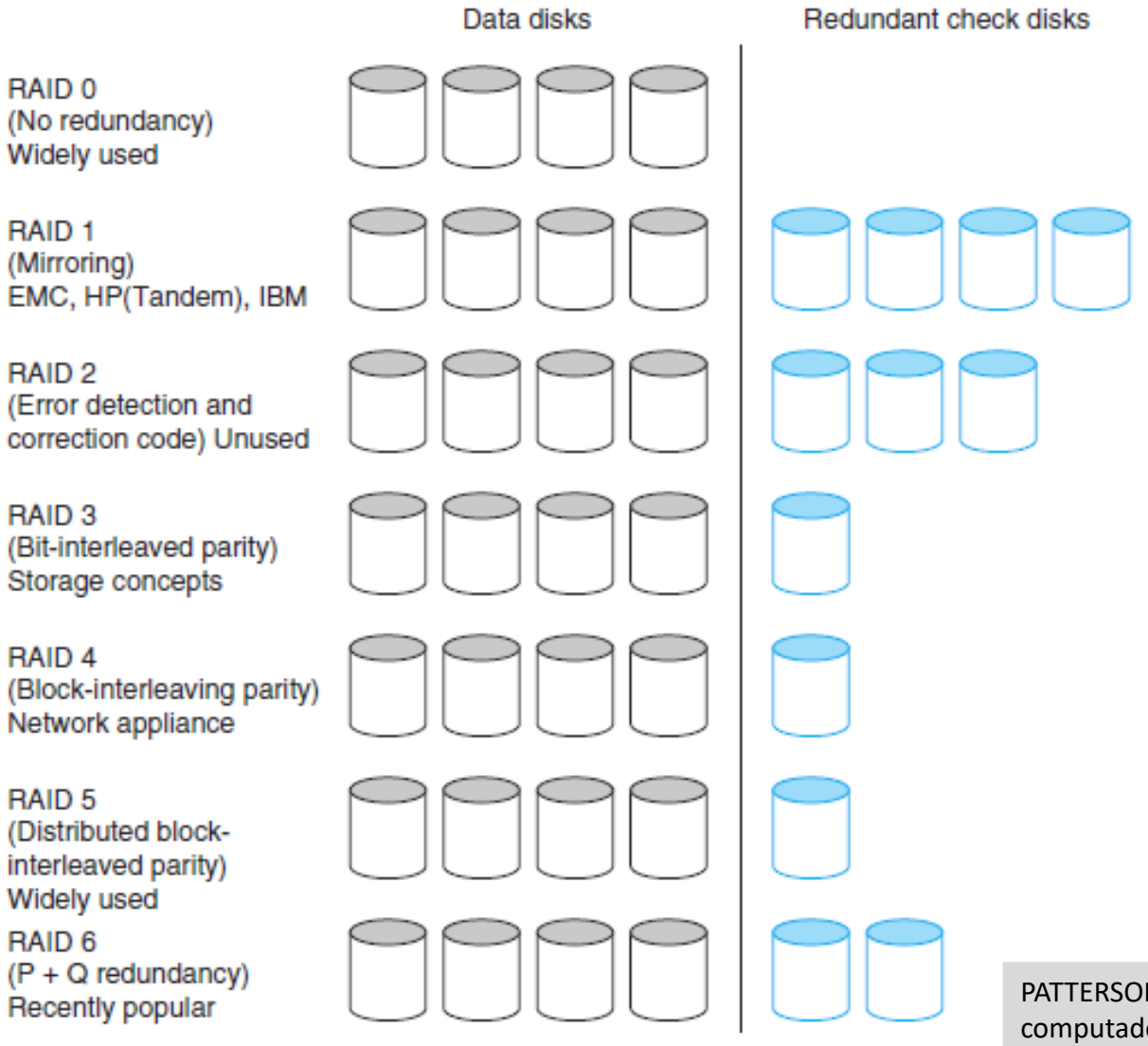
- TB = 1ms

- LRM = $\frac{0{,}5 \text{ rotação}}{5400 \text{ RPM}} = \frac{0{,}5 \text{ rotação}}{5400\text{RPM}/60(\text{seg})} = 0{,}0056\text{s} = 5{,}6\text{ms}$

- TT = 1kB / 50MB/s = $1*2^{10} / 50*10^{6}$ = 20,48µs

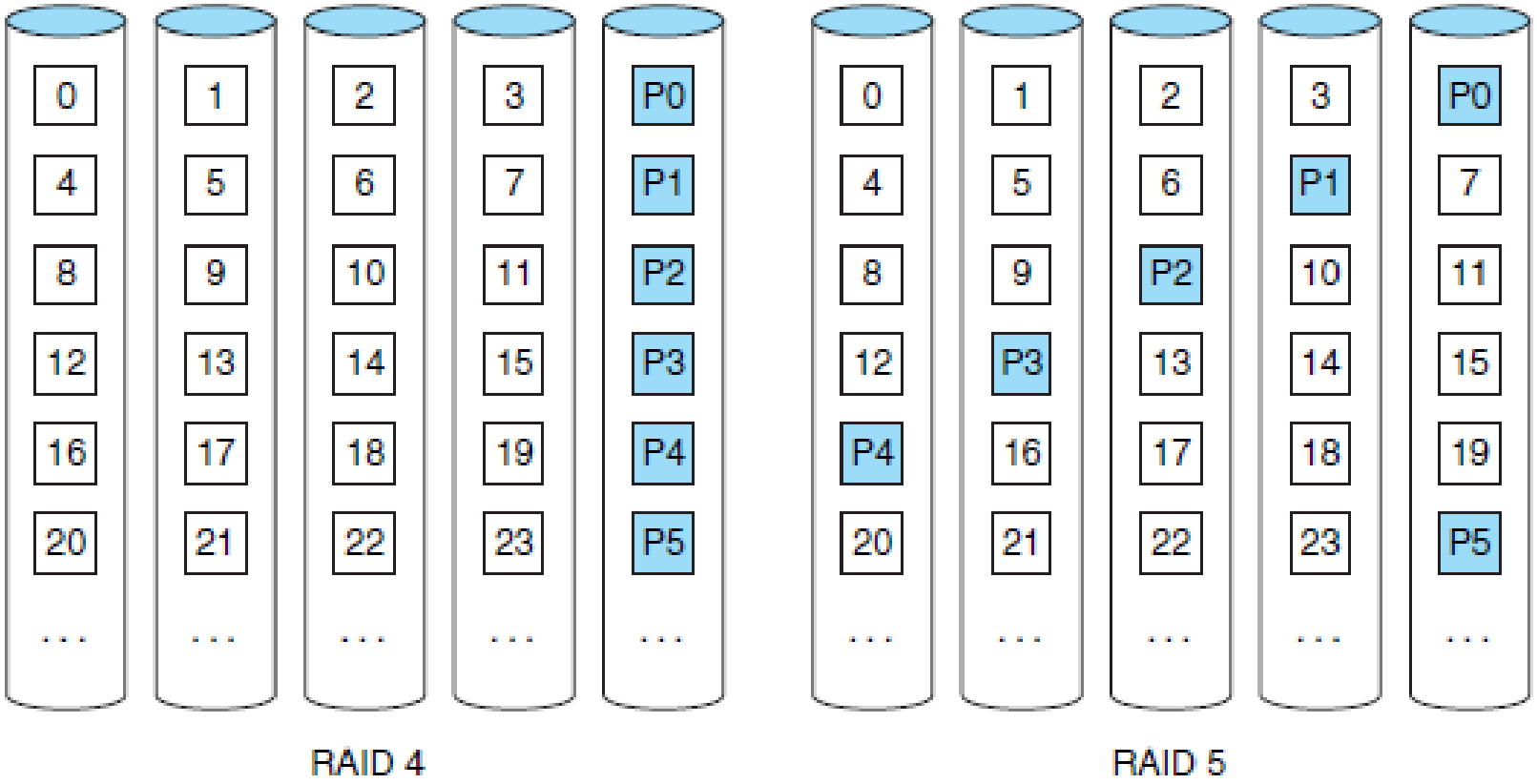
- TD = 1ms + 5,6ms + 0,02048ms = 6,62048ms

Redundant Arrays of Inexpensive Disks (RAID)



PATTERSON, David A. Organização e projeto de computadores a interface hardware/software

Redundant Arrays of Inexpensive Disks (RAID)



PATTERSON, David A. Organização e projeto de computadores a interface hardware/software